William Stallings
Computer Organization and Architecture
7th Edition

Chapter 1
Introduction

Architecture & Organization 1

- Architecture is those attributes visible to the programmer, those attributes that have a direct impact on the logical execution of a program
  - Instruction set, number of bits used for data representation, I/O mechanisms, addressing techniques.
  - e.g. Is there a multiply instruction?
- Organization is how features are implemented, hardware details transparent to the programmer
  - Control signals, interfaces with peripherals, memory technology.
  - e.g. Is there a hardware multiply unit or is it done by repeated addition?

Architecture & Organization 2

- All Intel x86 family share the same basic architecture
- The IBM System/370 family share the same basic architecture

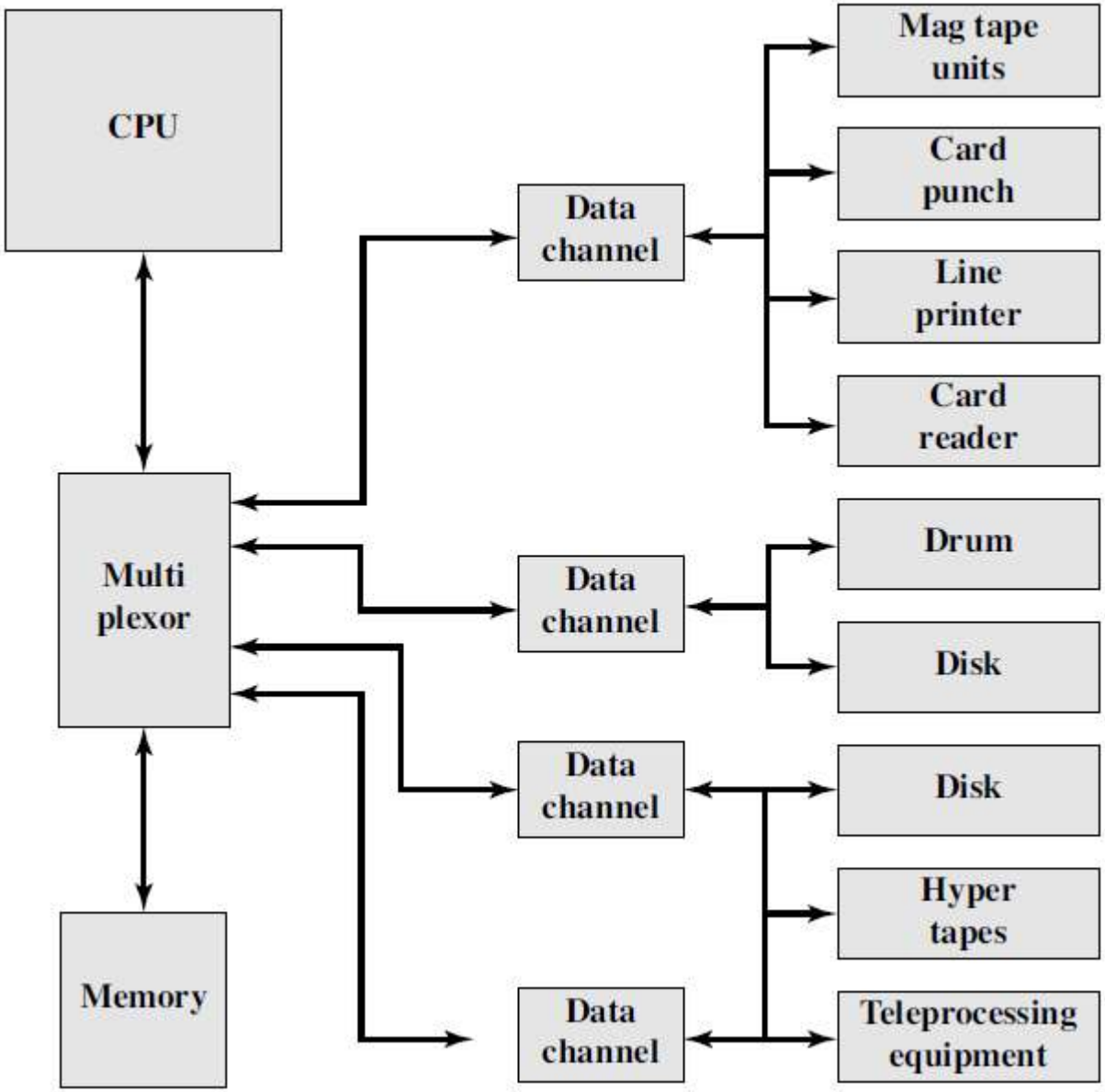
التوافق

- This gives code compatibility
  - At least backwards
- Organization differs between different versions

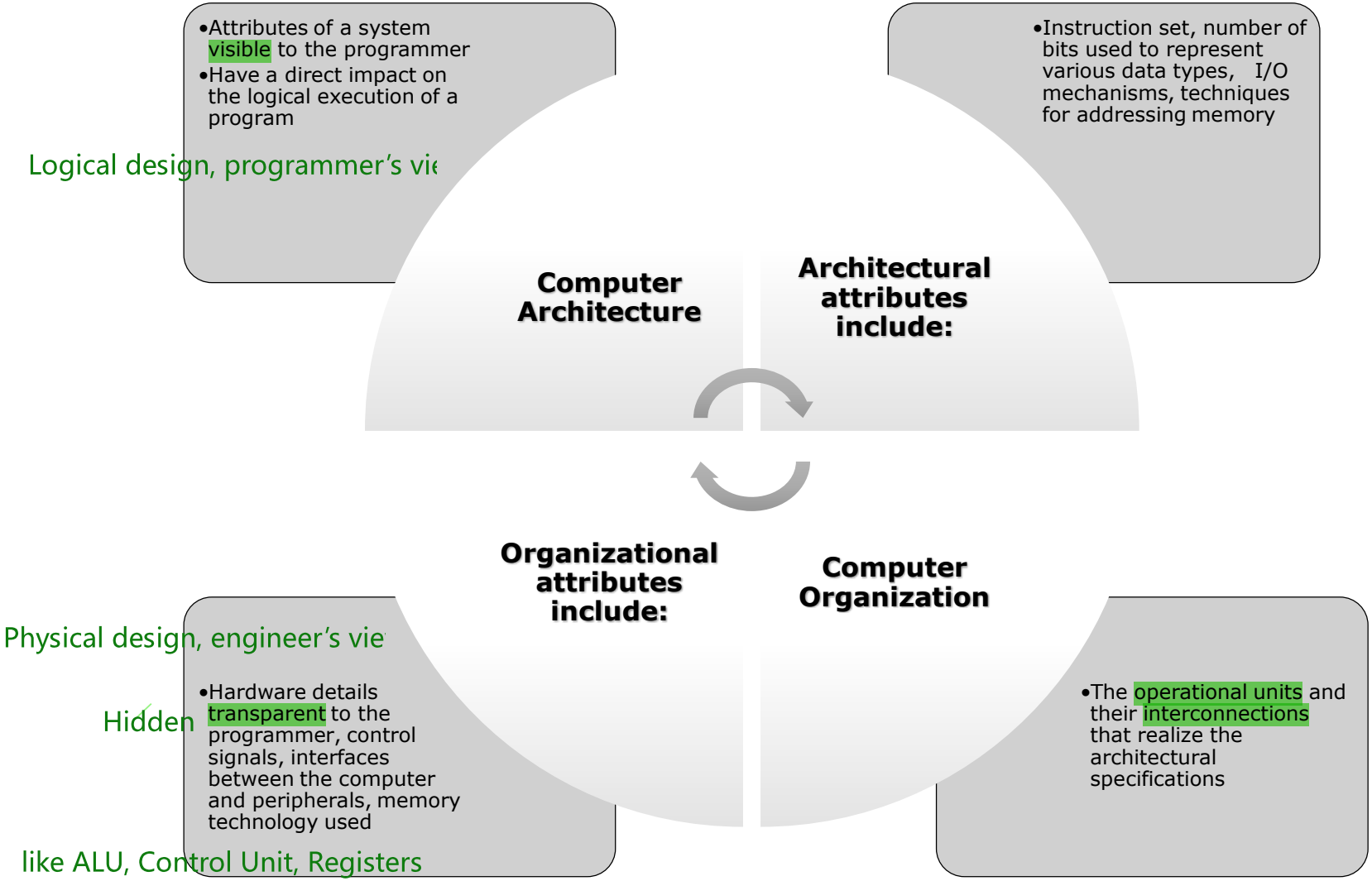
IBM System/370 Architecture

- Was introduced in 1970
- Included a number of models
- Could upgrade to a more expensive, faster model without having to abandon original software
- New models are introduced with improved technology, but retain the same architecture so that the customer's software investment is protected
- Architecture has survived to this day as the architecture of IBM's mainframe product line

An IBM 7094 Configuration



Computer Architecture|Computer Organization



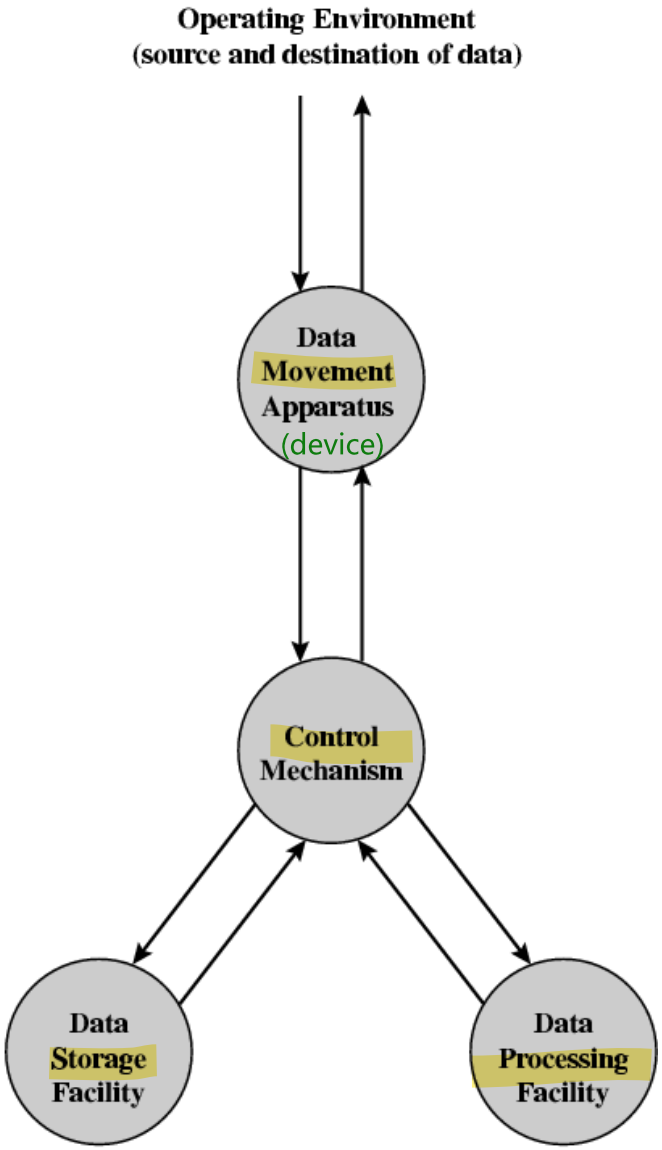
Structure & Function

- Structure is the way in which components relate to each other
- Function is the operation of individual components as part of the structure
- Hierarchical system
  - Set of interrelated subsystems
- Hierarchical nature of complex systems is essential to both their design and their description
- Designer need only deal with a particular level of the system at a time
  - Concerned with structure and function at each level

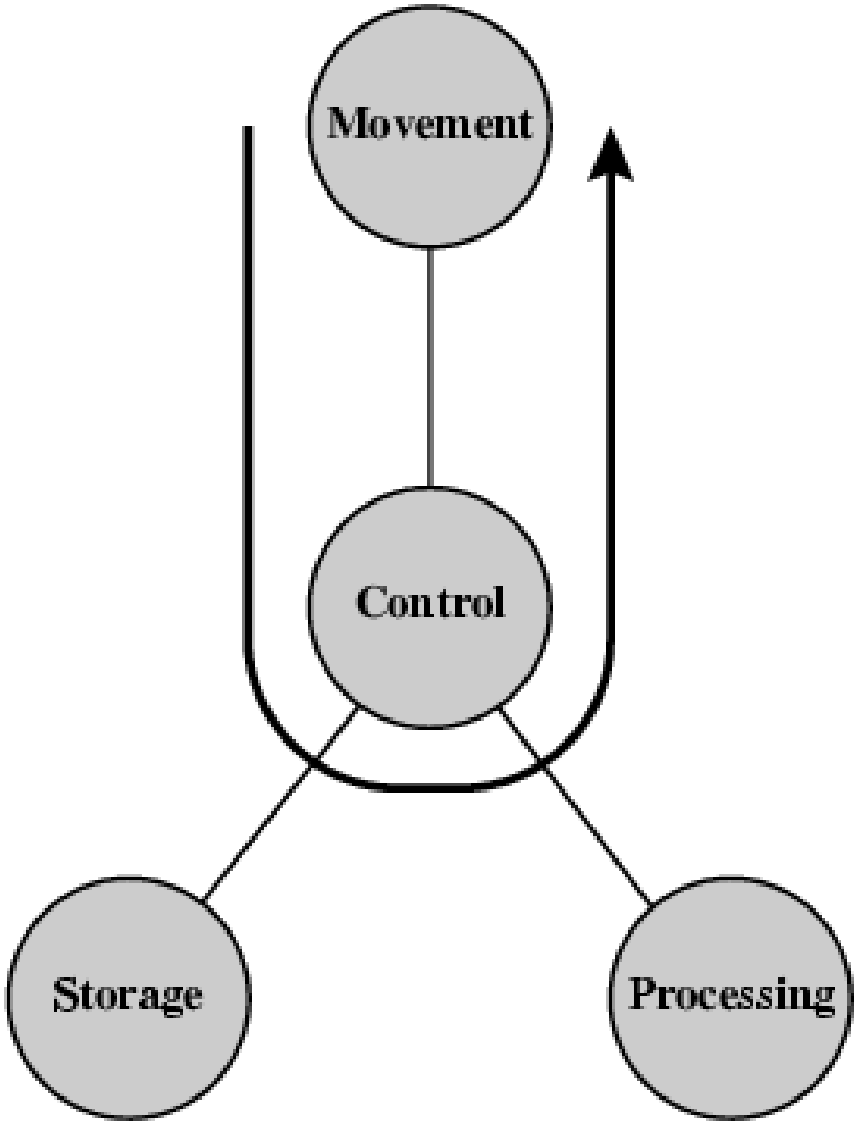
Function

- All computer functions are:
- Data processing: Data may take a wide variety of forms and the range of processing requirements is broad
- Data storage: Short-term, Long-term
- Data movement
  - Input-output (I/O) - when data are received from or delivered to a device (peripheral) that is directly connected to the computer
  - Data communications – when data are moved over longer distances, to or from a remote device
- Control: A control unit manages the computer's resources and orchestrates the performance of its functional parts in response to instructions

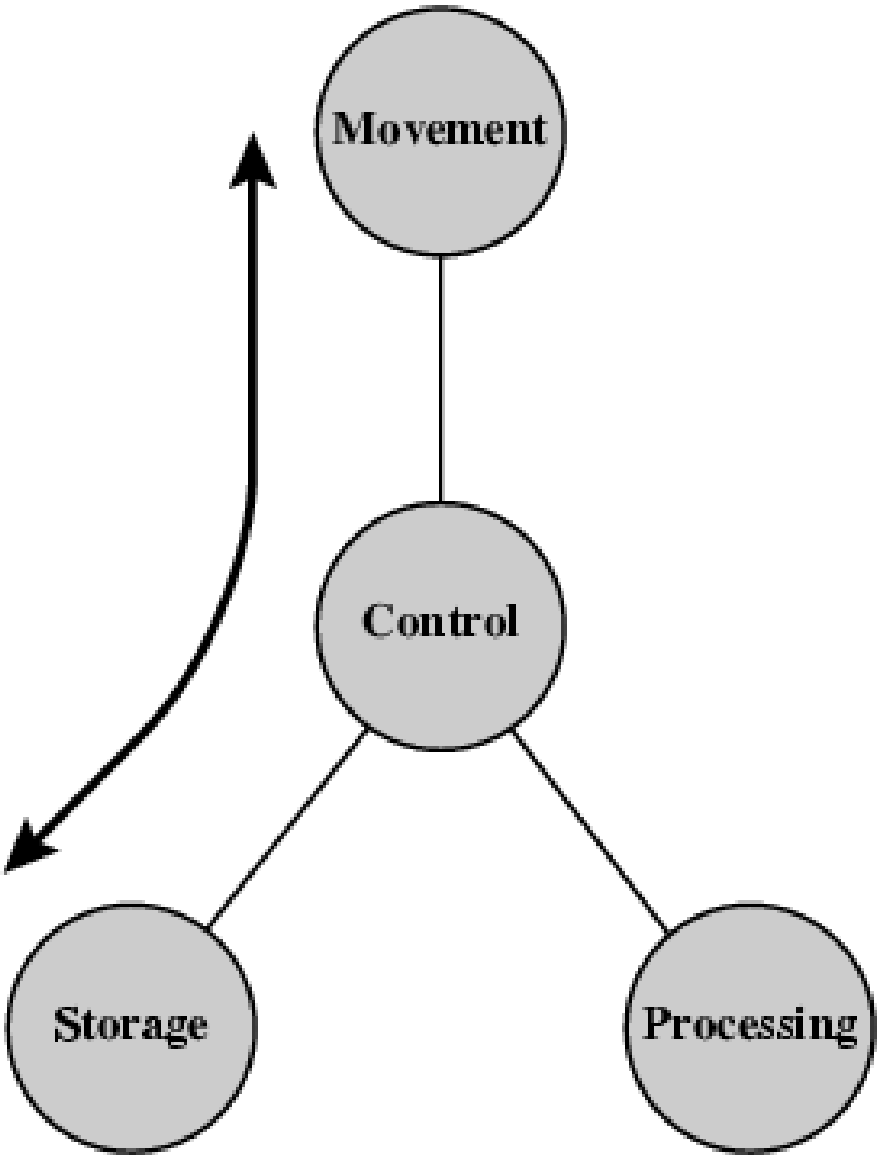
Functional View



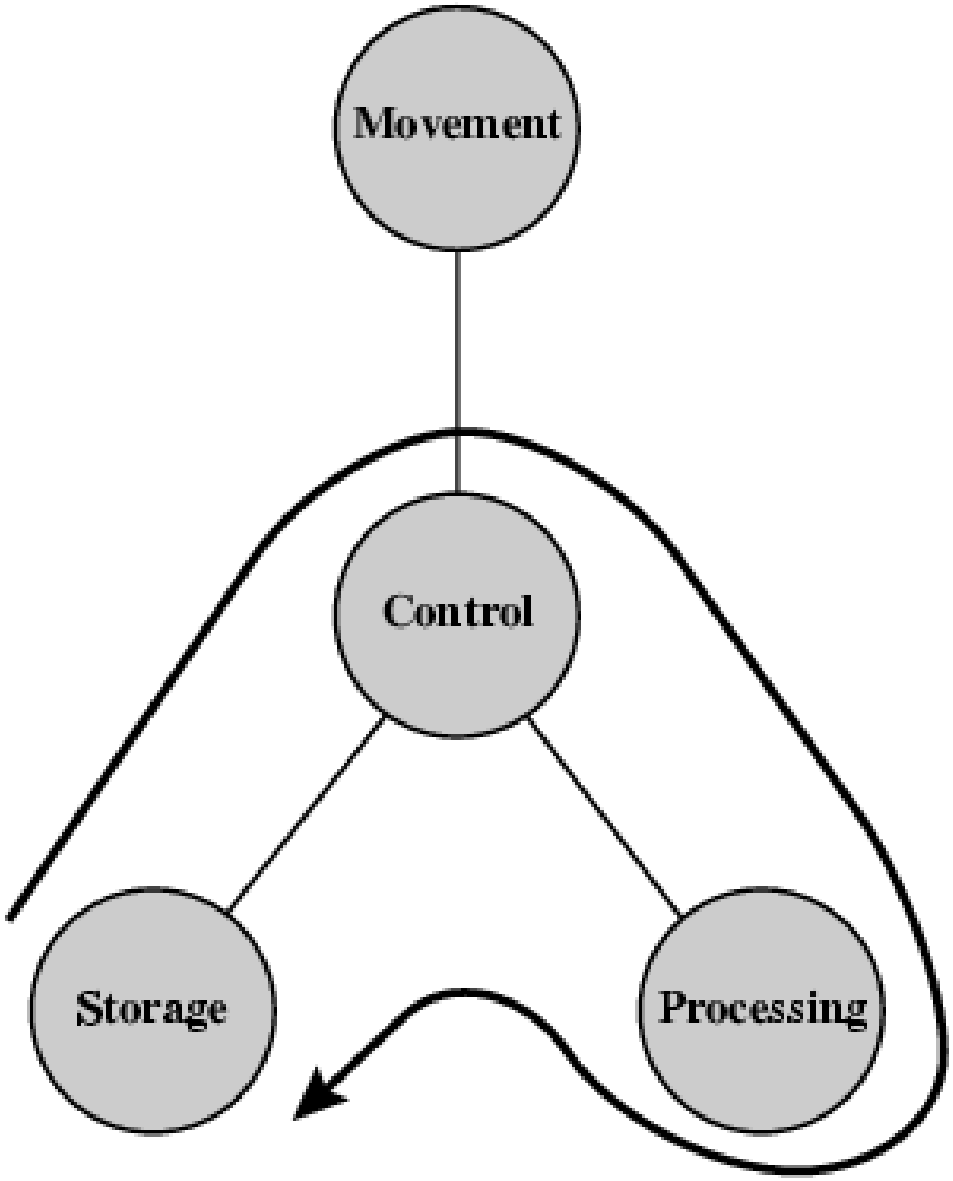
Operations (a) Data movement



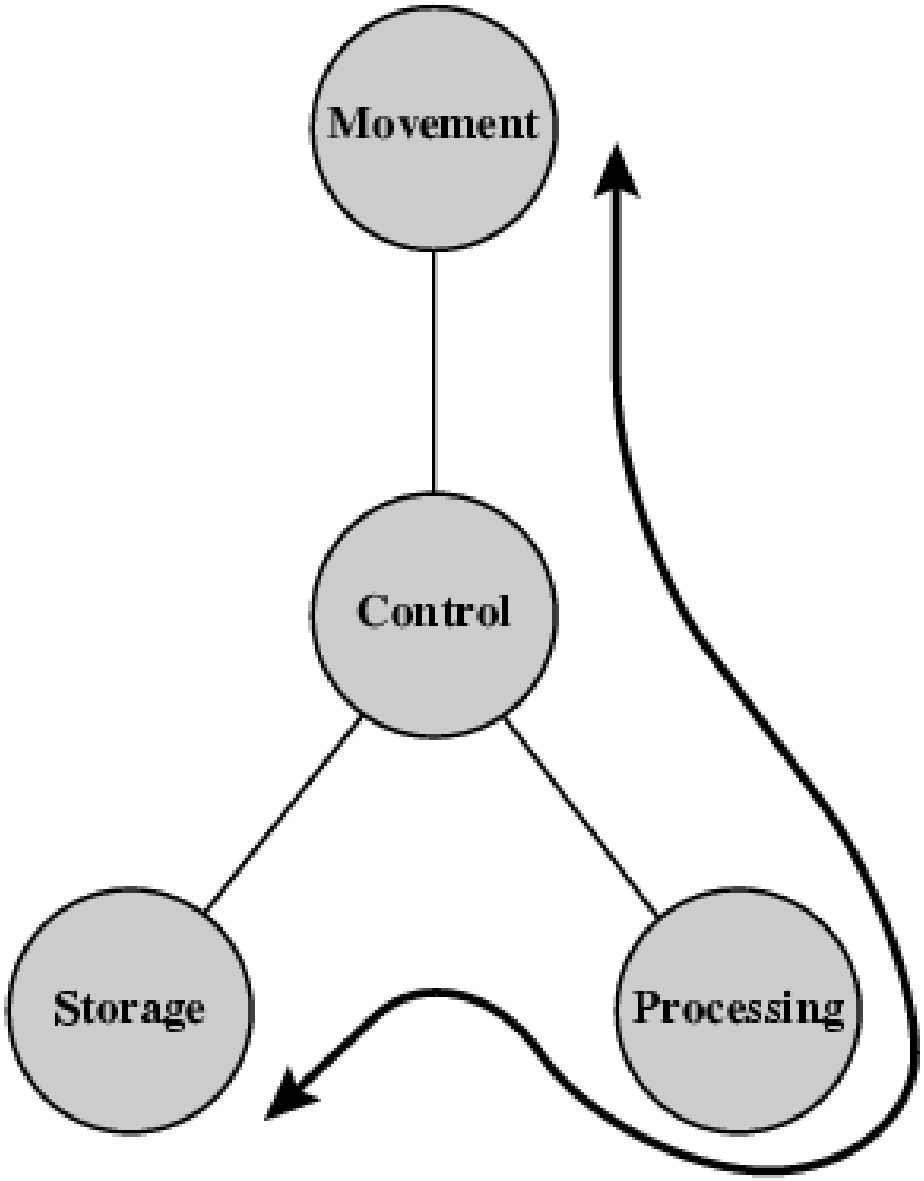
Operations (b) Storage



Operation (c) Processing from/to storage



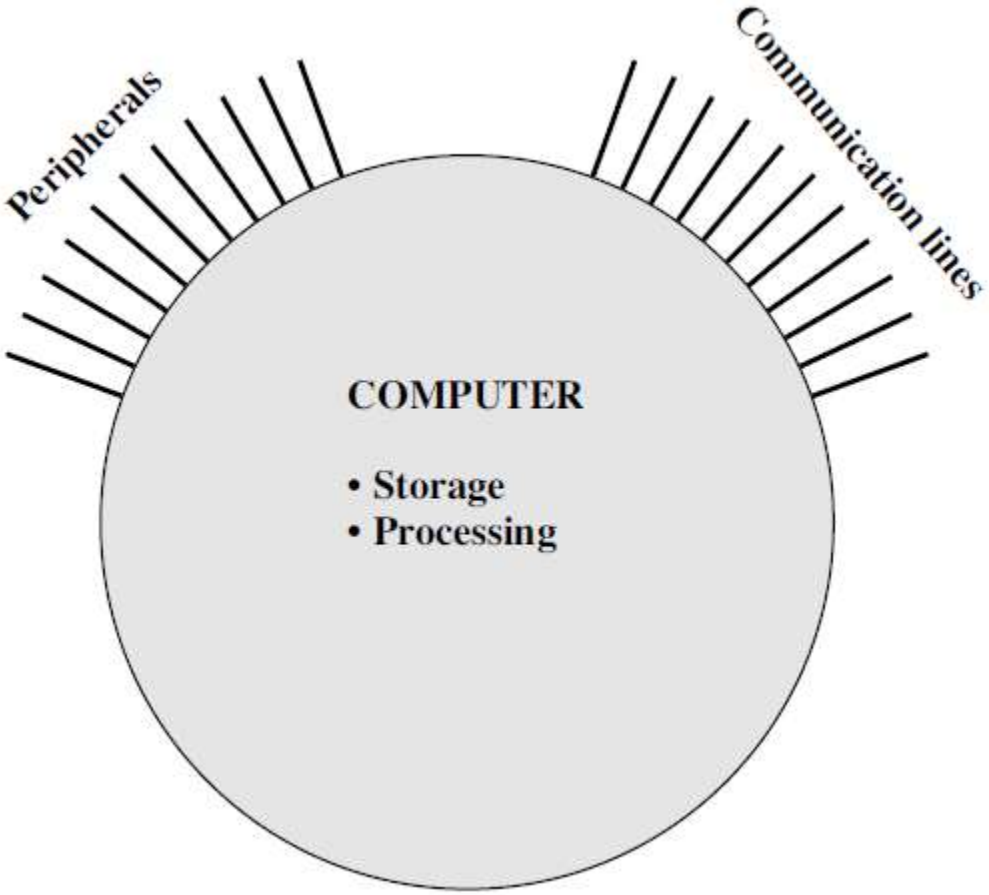
Operation (d)
Processing from storage to I/O



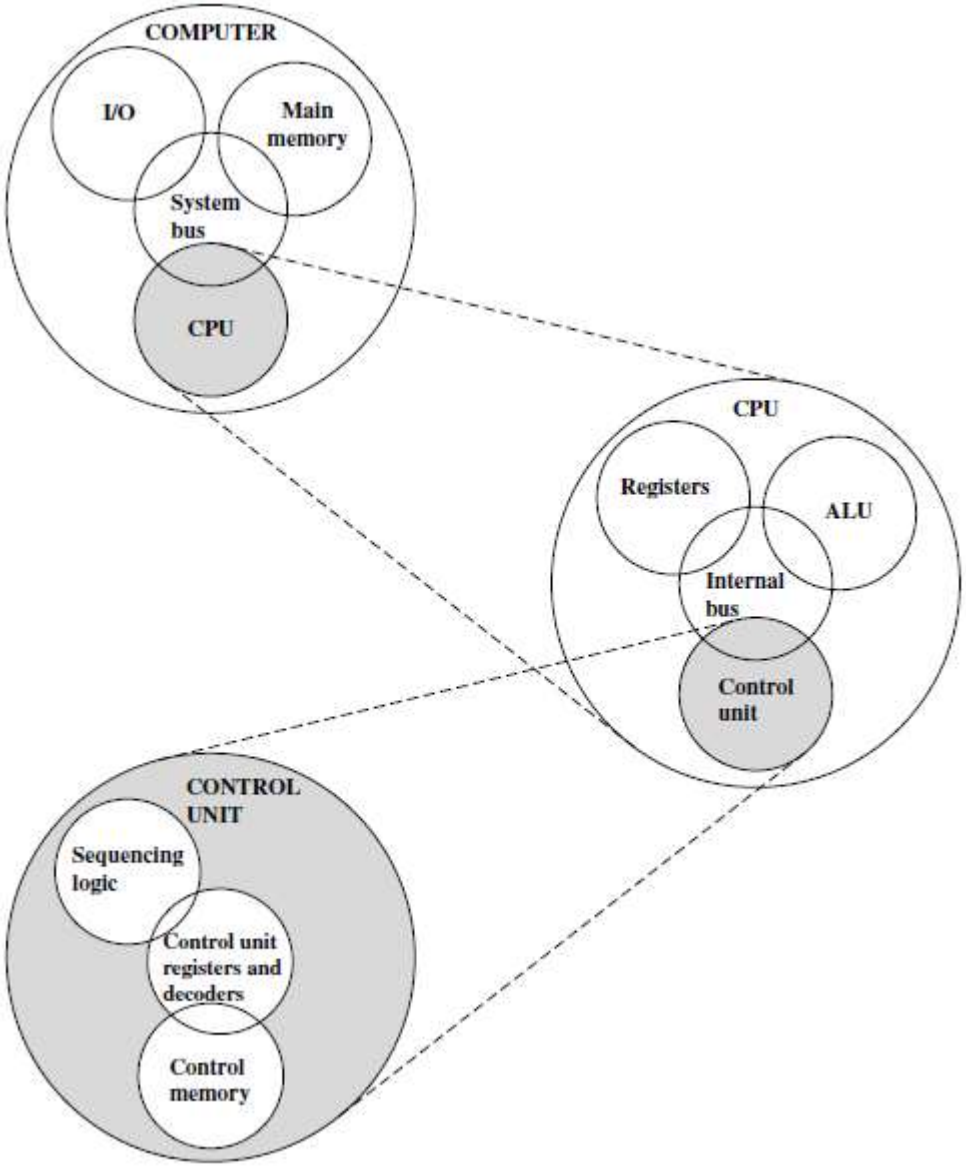
The Computer

somehow

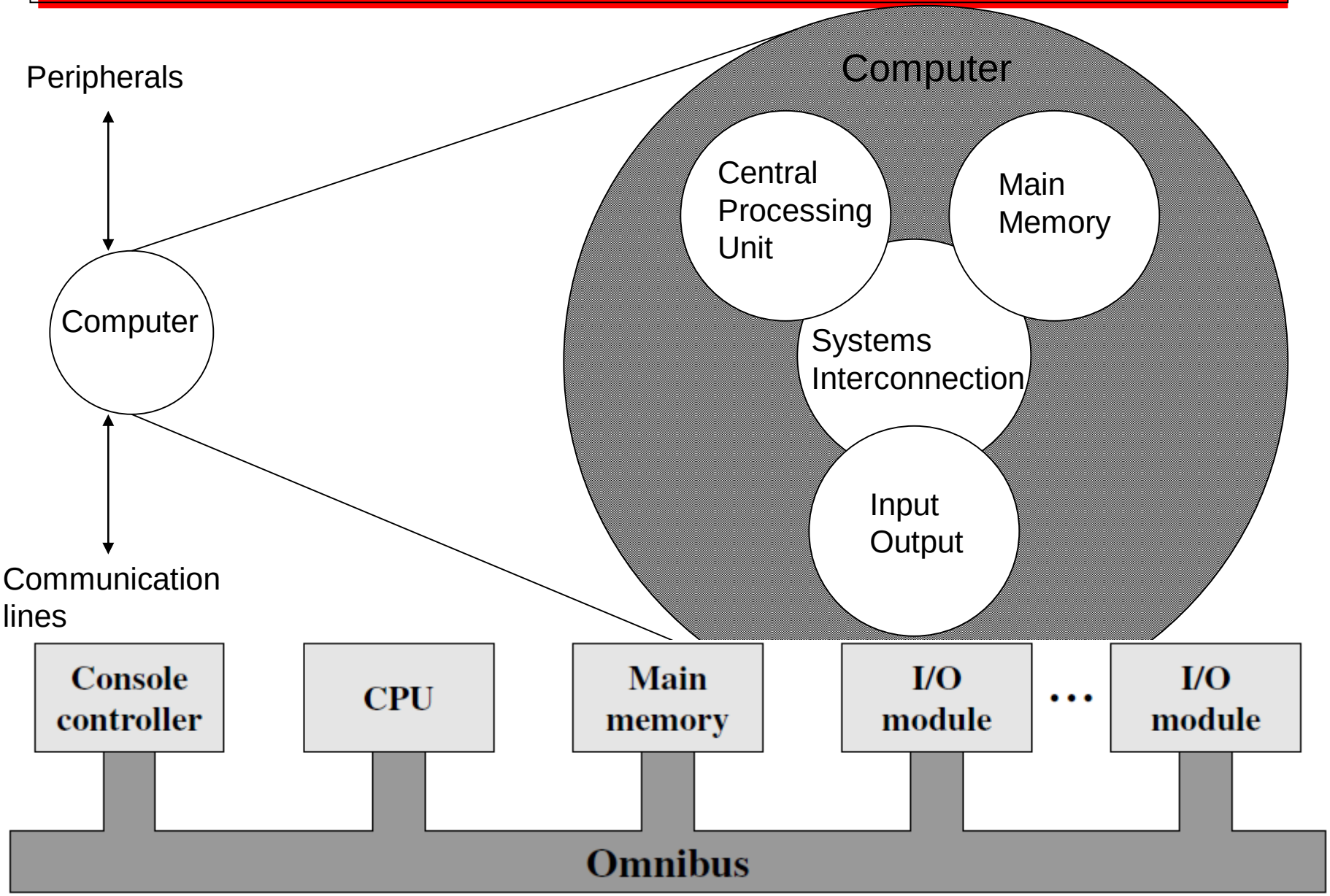
The computer interacts in some fashion with its external environment. In general, all of its linkages to the external environment can be classified as peripheral devices or communication lines.



Structure


COMPUTER
I/O
Main memory
System bus
CPU
CPU
Registers
ALU
Internal bus
Control unit
CONTROL UNIT
Sequencing logic
Control unit registers and decoders
Control memory

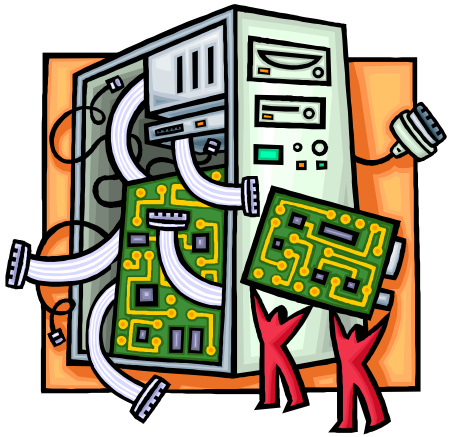
Structure - Top Level



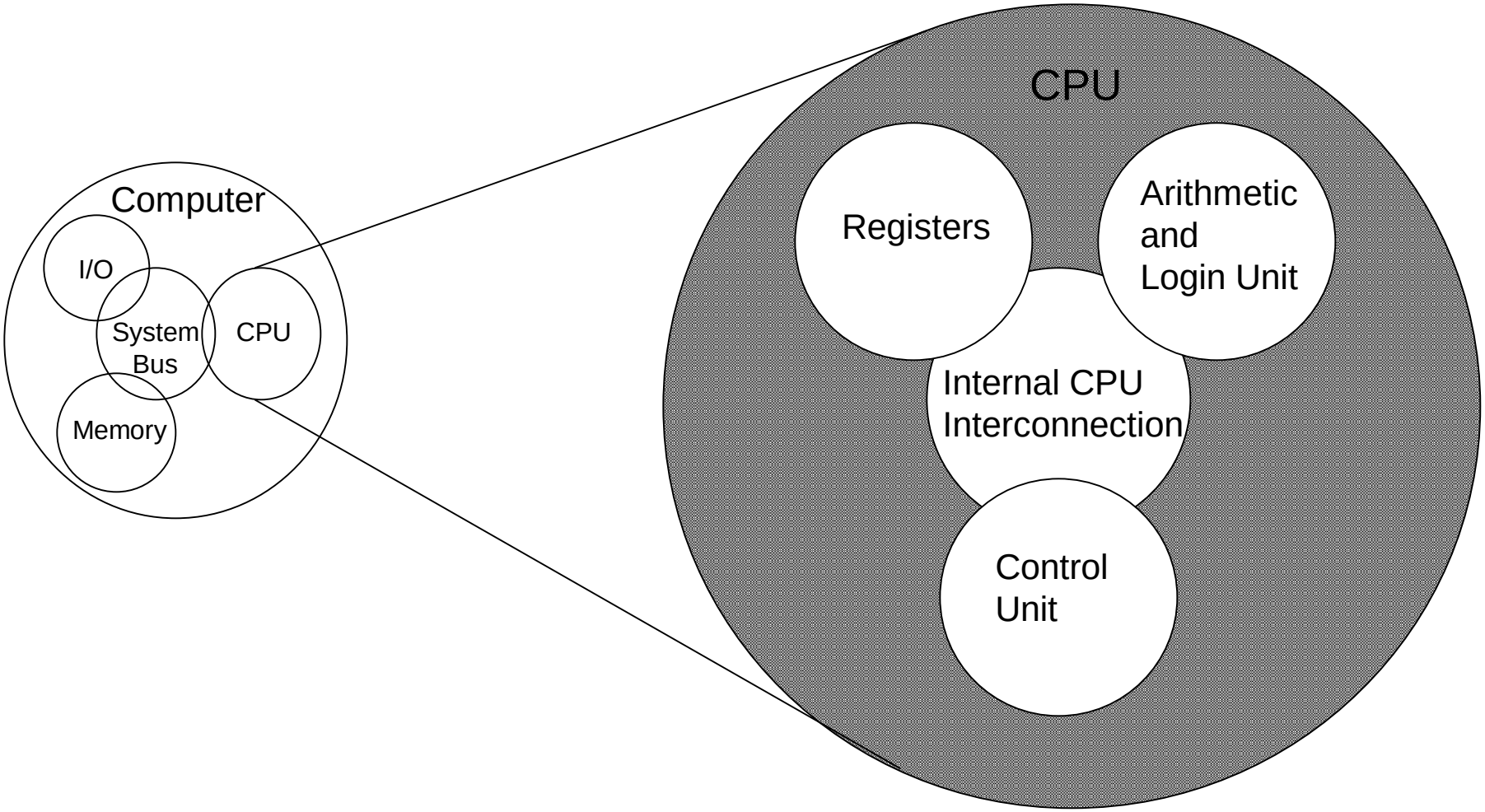
Structural Components of The Computer

There are four main structural components of the computer:

- CPU – controls the operation of the computer and performs its data processing functions
- Main Memory – stores data
- I/O – moves data between the computer and its external environment
- System Interconnection – some mechanism that provides for communication among CPU, main memory, and I/O



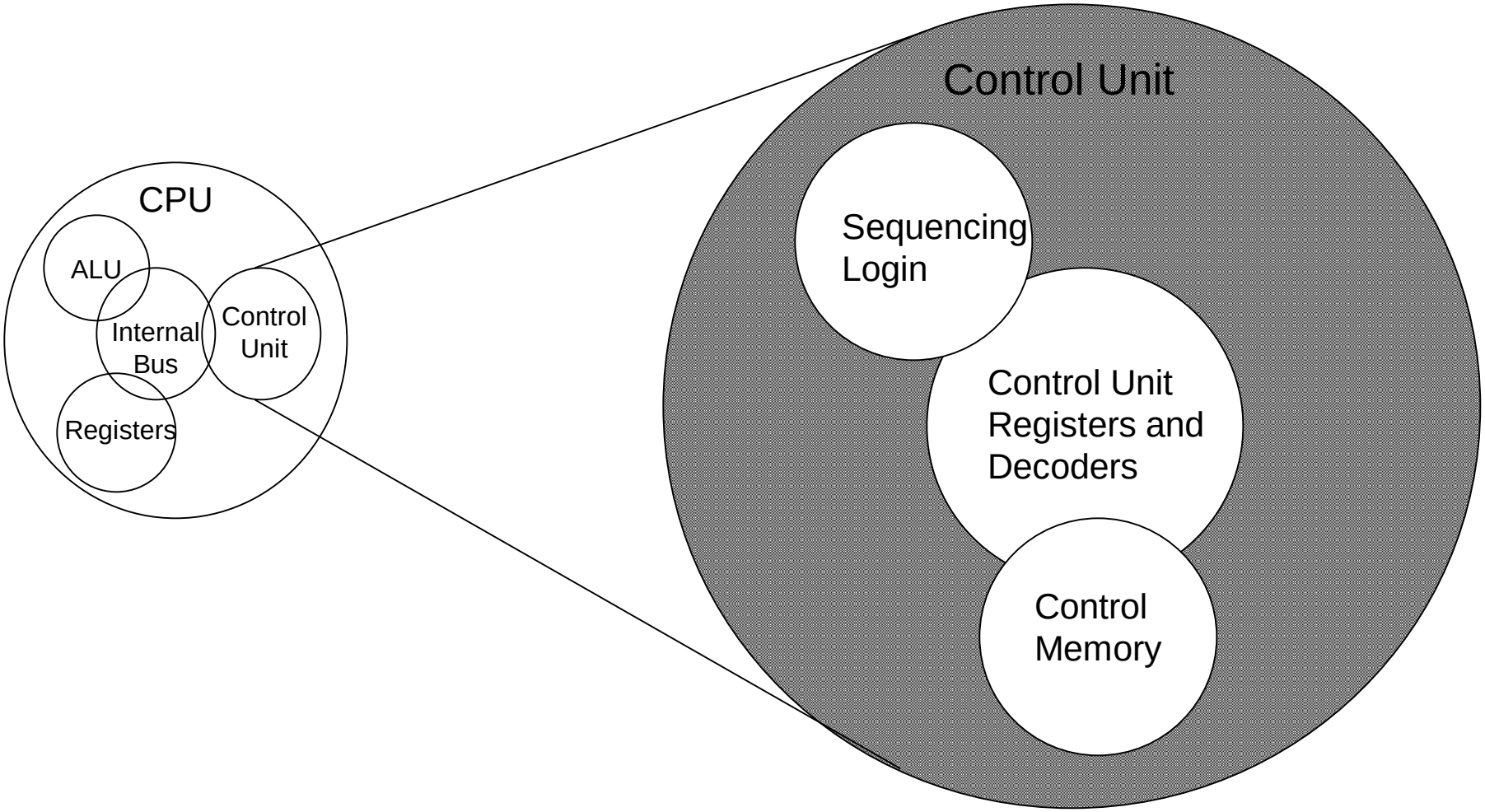
Structure - The CPU



CPU: Major structural components

- Control Unit
  - Controls the operation of the CPU and hence the computer
- Arithmetic and Logic Unit (ALU)
  - Performs the computer's data processing function
- Registers
  - Provide storage internal to the CPU
- CPU Interconnection
  - Some mechanism that provides for communication among the control unit, ALU, and registers

Structure - The Control Unit



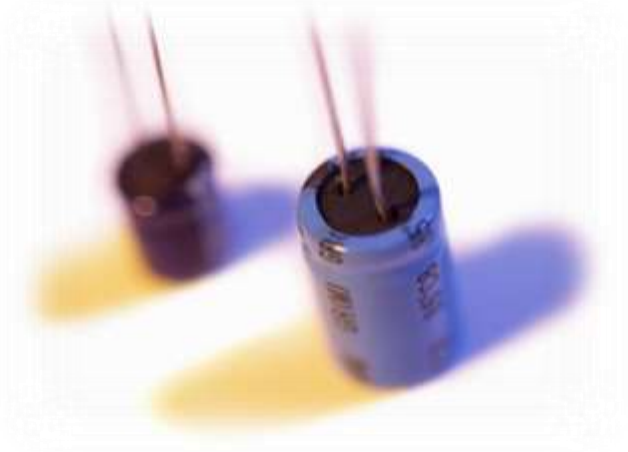
Vacuum Tubes

- Vacuum tubes were used for digital logic elements and memory

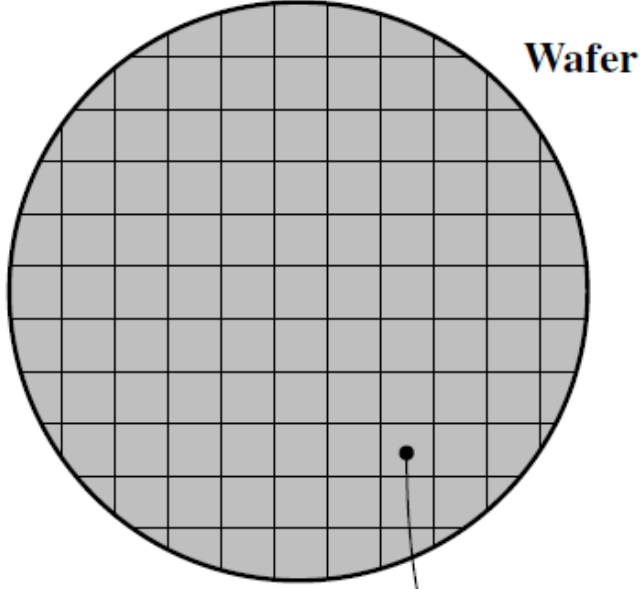
- IAS computer: (Institute for Advanced Study)
  - Fundamental design approach was the stored program concept
    - Attributed to the mathematician John von Neumann
    - First publication of the idea was in 1945 for the EDVAC
  - Design began at the Princeton Institute for Advanced Studies
  - Completed in 1952
  - Prototype of all subsequent general-purpose computers

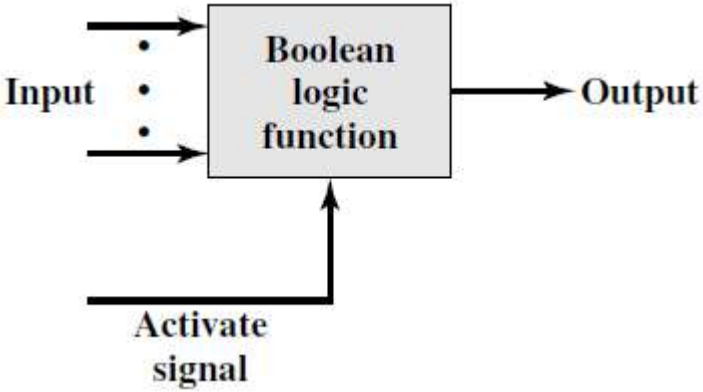
Transistors

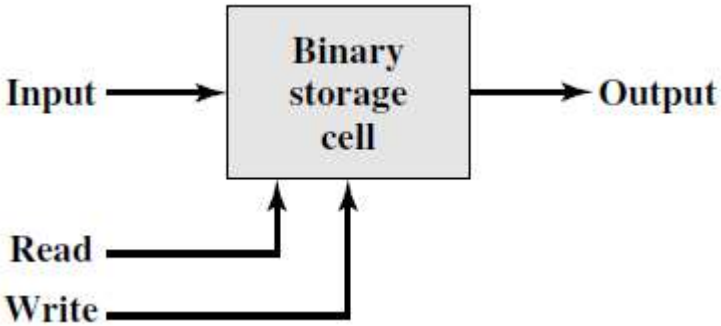
- Smaller
- Cheaper
- Dissipates less heat than a vacuum tube
- Is a solid state device made from silicon
- Was invented at Bell Labs in 1947
- It was not until the late 1950's that fully transistorized computers were commercially available

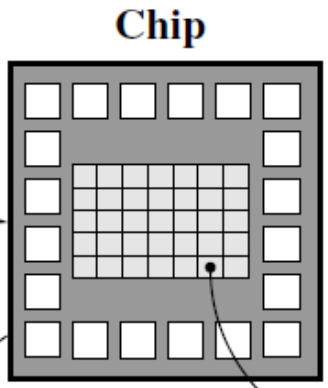


Chips

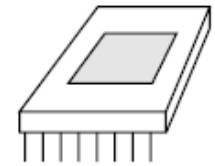

Wafer


Input
Boolean logic function
Output
Activate signal

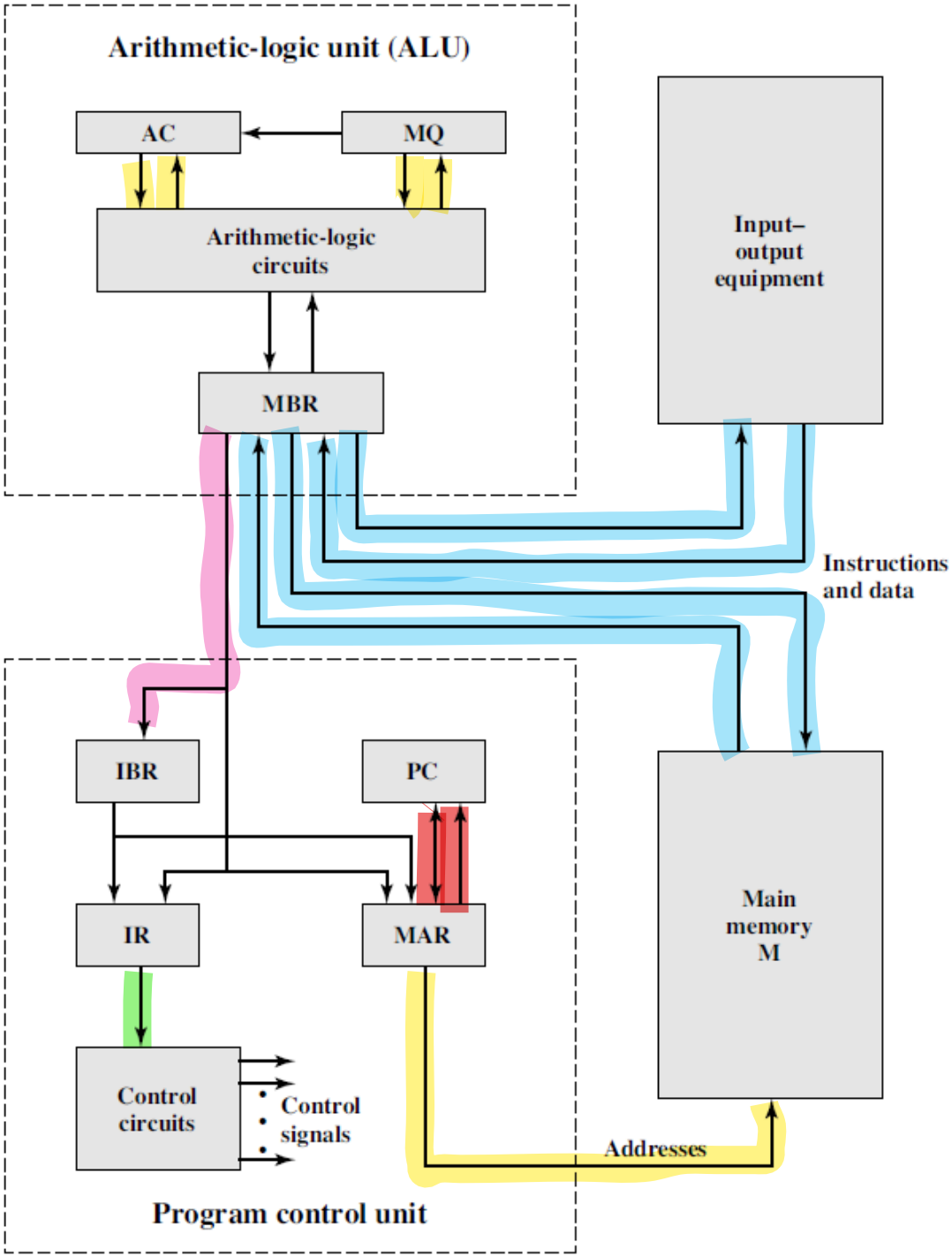

Input
Binary storage cell
Output
Read
Write


Chip

Gate


Packaged chip

Instruction Architecture Set



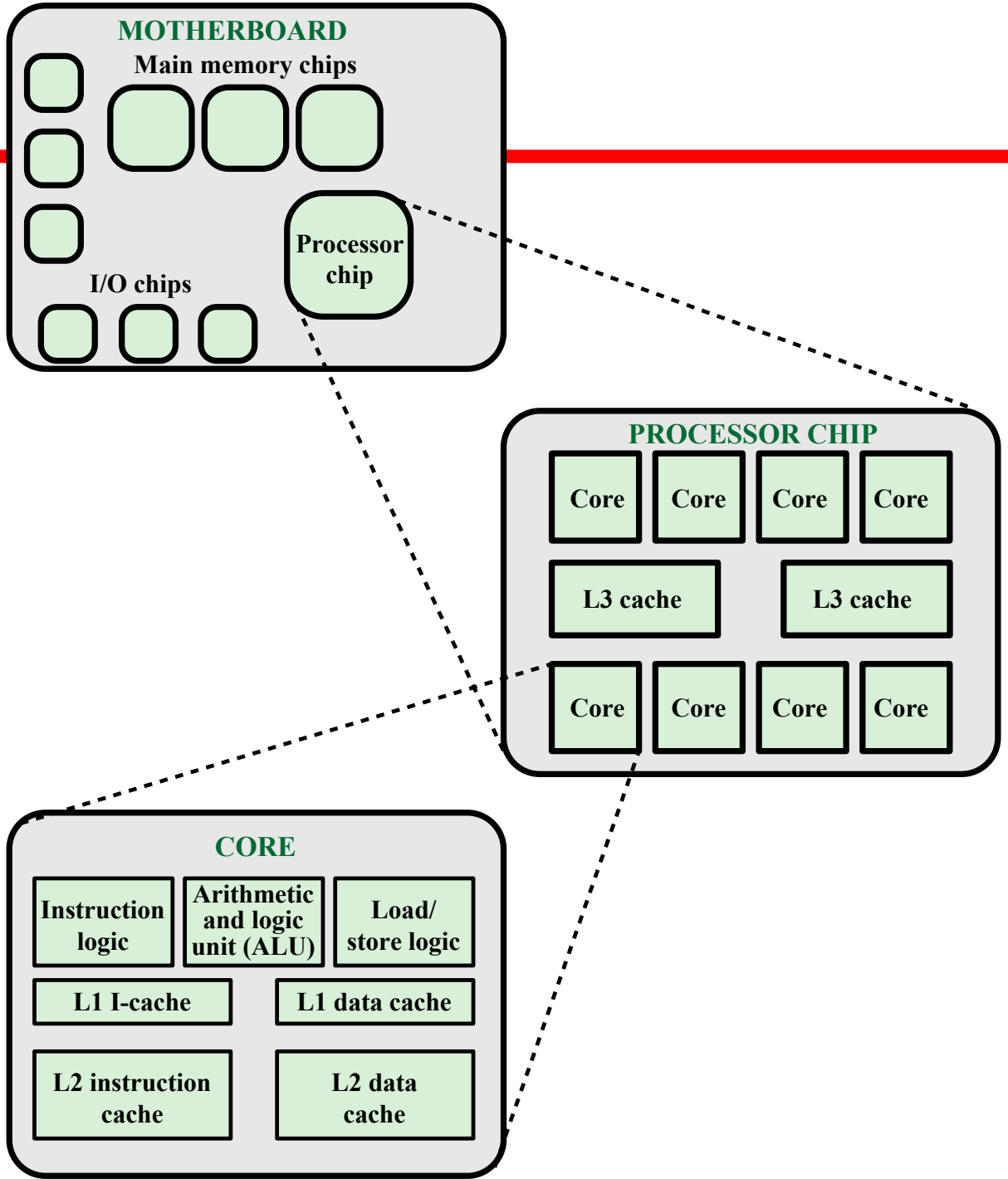
Registers

Memory buffer register (MBR)	•Contains a word to be stored in memory or sent to the I/O unit •Or is used to receive a word from memory or from the I/O unit
Memory address register (MAR)	•Specifies the address in memory of the word to be written from or read into the MBR
Instruction register (IR)	•Contains the 8-bit opcode instruction being executed
Instruction buffer register (IBR)	•Employed to temporarily hold the right-hand instruction from a word in memory
Program counter (PC)	•Contains the address of the next instruction pair to be fetched from memory
Accumulator (AC) and multiplier quotient (MQ)	•Employed to temporarily hold operands and results of ALU operations

Multicore Computer Structure

- Central processing unit (CPU)
  - Portion of the computer that fetches and executes instructions
  - Consists of an ALU, a control unit, and registers
  - Referred to as a processor in a system with a single processing unit
- Core
  - An individual processing unit on a processor chip
  - May be equivalent in functionality to a CPU on a single-CPU system
  - Specialized processing units are also referred to as cores
- Processor
  - A physical piece of silicon containing one or more cores
  - Is the computer component that interprets and executes instructions
  - Referred to as a *multicore processor* if it contains multiple cores

Multicore


MOTHERBOARD
Main memory chips
Processor chip
I/O chips
PROCESSOR CHIP
Core
Core
Core
Core
L3 cache
L3 cache
Core
Core
Core
Core
CORE
Instruction logic
Arithmetic and logic unit (ALU)
Load/ store logic
L1 I-cache
L1 data cache
L2 instruction cache
L2 data cache

Cache Memory

- Multiple layers of memory between the processor and main memory
- Is smaller and faster than main memory
- Used to speed up memory access by placing in the cache data from main memory that is likely to be used in the near future
- A greater performance improvement may be obtained by using multiple levels of cache, with level 1 (L1) closest to the core and additional levels (L2, L3, etc.) progressively farther from the core

Outline of the Book (1)

- Computer Evolution and Performance
- Computer Interconnection Structures
- Internal Memory
- External Memory
- Input/Output
- Operating Systems Support
- Computer Arithmetic
- Instruction Sets

Outline of the Book (2)

- CPU Structure and Function
- Reduced Instruction Set Computers
- Superscalar Processors
- Control Unit Operation
- Microprogrammed Control
- Multiprocessors and Vector Processing
- Digital Logic (Appendix)

Internet Resources
- Web site for book

- http://WilliamStallings.com/COA/COA7e.html
  - links to sites of interest
  - links to sites for courses that use the book
  - errata list for book
  - information on other books by W. Stallings
- http://WilliamStallings.com/StudentSupport.html
  - Math
  - How-to
  - Research resources
  - Misc

Internet Resources
- Web sites to look for

- WWW Computer Architecture Home Page
- CPU Info Center
- Processor Emporium
- ACM Special Interest Group on Computer Architecture
- IEEE Technical Committee on Computer Architecture
- Intel Technology Journal
- Manufacturer's sites
  - Intel, IBM, etc.

Internet Resources
- Usenet News Groups

- comp.arch
- comp.arch.arithmetic
- comp.arch.storage
- comp.parallel

William Stallings
Computer Organization and Architecture
7th Edition

Input/Output

Input/Output Problems

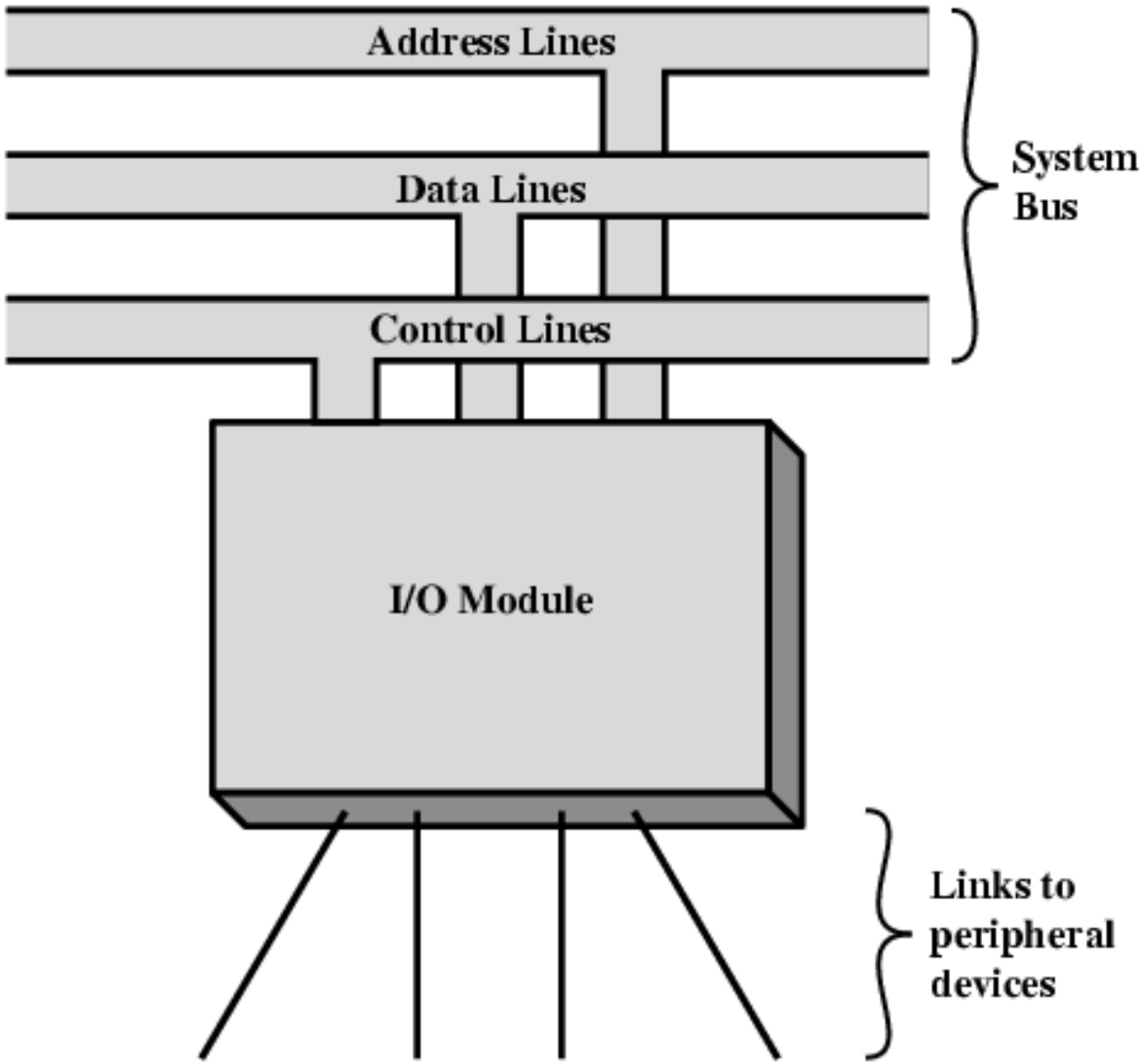
why one does not connect peripherals directly to the system bus

- Wide **variety** of peripherals: There are a wide variety of peripherals with various methods of operation.
- All **slower** than CPU and RAM:
  - The data transfer rate of peripherals is often much slower than that of the memory or processor.
  - On the other hand, the data transfer rate of some peripherals is faster than that of the memory or processor.
- Peripherals often use different data **formats** and word lengths than the computer to which they are attached.

Input/Output Module

- Need I/O modules: Each module interfaces to the system bus or central switch and controls one or more peripheral devices.
  - It contains logic for performing a **communication function** between the peripheral and the bus to exchange control, status, and data
- Thus, the I/O module has two major functions
  - Interface to the processor and memory via the system bus or central switch
  - Interface to one or more peripheral devices by tailored data links

Generic Model of I/O Module



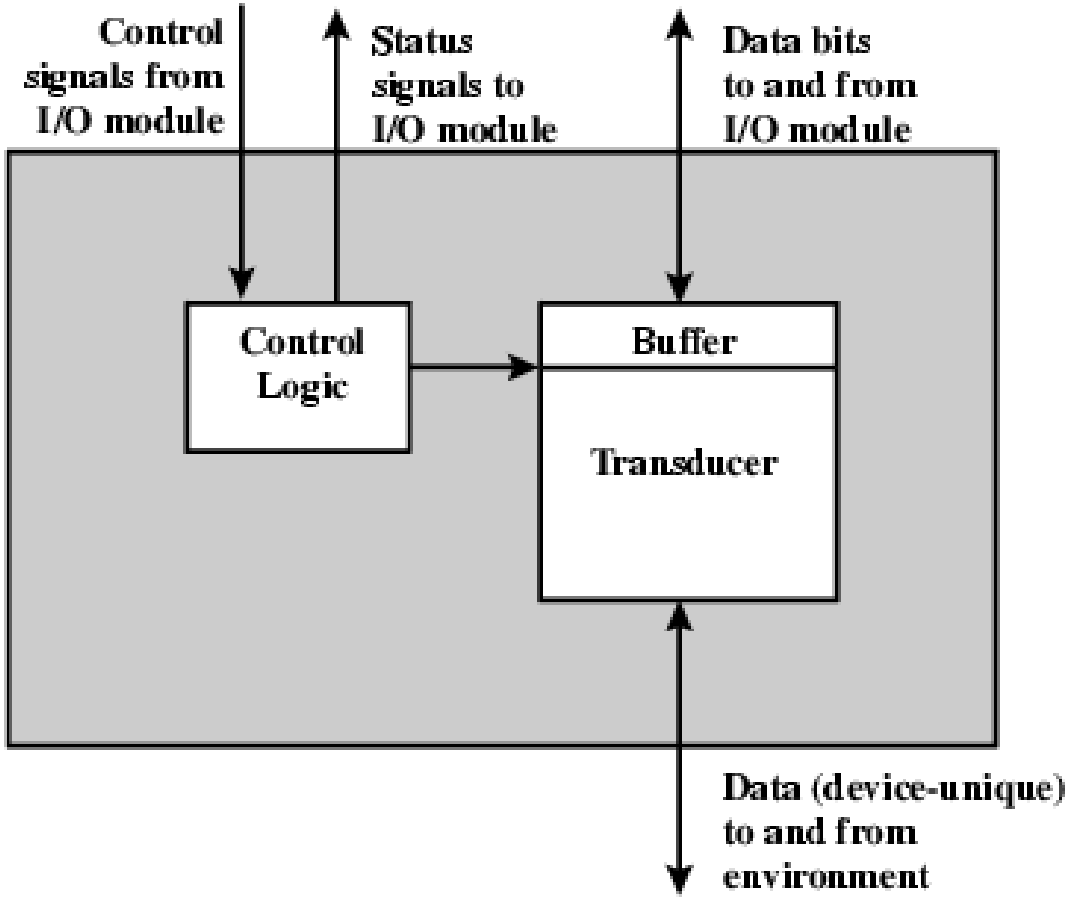
External Devices: means of exchanging data between the external environment and the computer

- Human readable: Suitable for communicating with the computer user
  - Screen, printer, keyboard, Video display terminals (VDTs)
- Machine readable: Suitable for communicating with equipment
  - Monitoring and control: Magnetic disk and tape systems, sensors and actuators
- Communication: Suitable for communicating with remote devices such as a terminal, a machine readable device, or another computer
  - Modem, Network Interface Card (NIC)

External Device Block Diagram

Control signals determine the function that the device will perform, such as send data to the I/O module (INPUT or READ), accept data from the I/O module (OUTPUT or WRITE), report status, or perform some control function particular to the device (e.g., position a disk head). Data are in the form of a set of bits to be sent to or received from the I/O module. Status signals indicate the state of the device. Examples are READY/NOT-READY to show whether the device is ready for data transfer.

Control logic associated with the device controls the device's operation in response to direction from the I/O module. The transducer converts data from electrical to other forms of energy during output and from other forms to electrical during input. Typically, a buffer is associated with the transducer to temporarily hold data being transferred between the I/O module and the external environment. A buffer size of 8 to 16 bits is common for serial devices, whereas block- oriented devices such as disk drive controllers may have much larger buffers.



Keyboard/Monitor

Most common means of computer/user interaction

User provides input through the keyboard

The monitor displays data provided by the computer

- International Reference Alphabet (IRA)

- Basic unit of exchange is the character
  - Associated with each character is a code
  - Each character in this code is represented by a unique 7-bit binary code
    - 128 different characters can be represented
- Characters are of two types:
  - Printable
    - Alphabetic, numeric, and special characters that can be printed on paper or displayed on a screen
  - Control
    - Have to do with controlling the printing or displaying of characters
    - Example is carriage return
    - Other control characters are concerned with communications procedures

- Keyboard Codes

- When the user depresses a key it generates an electronic signal that is interpreted by the transducer in the keyboard and translated into the bit pattern of the corresponding IRA code

- This bit pattern is transmitted to the I/O module in the computer

- On output, IRA code characters are transmitted to an external device from the I/O module

- The transducer interprets the code and sends the required electronic signals to the output device either to display the indicated character or perform the requested control function

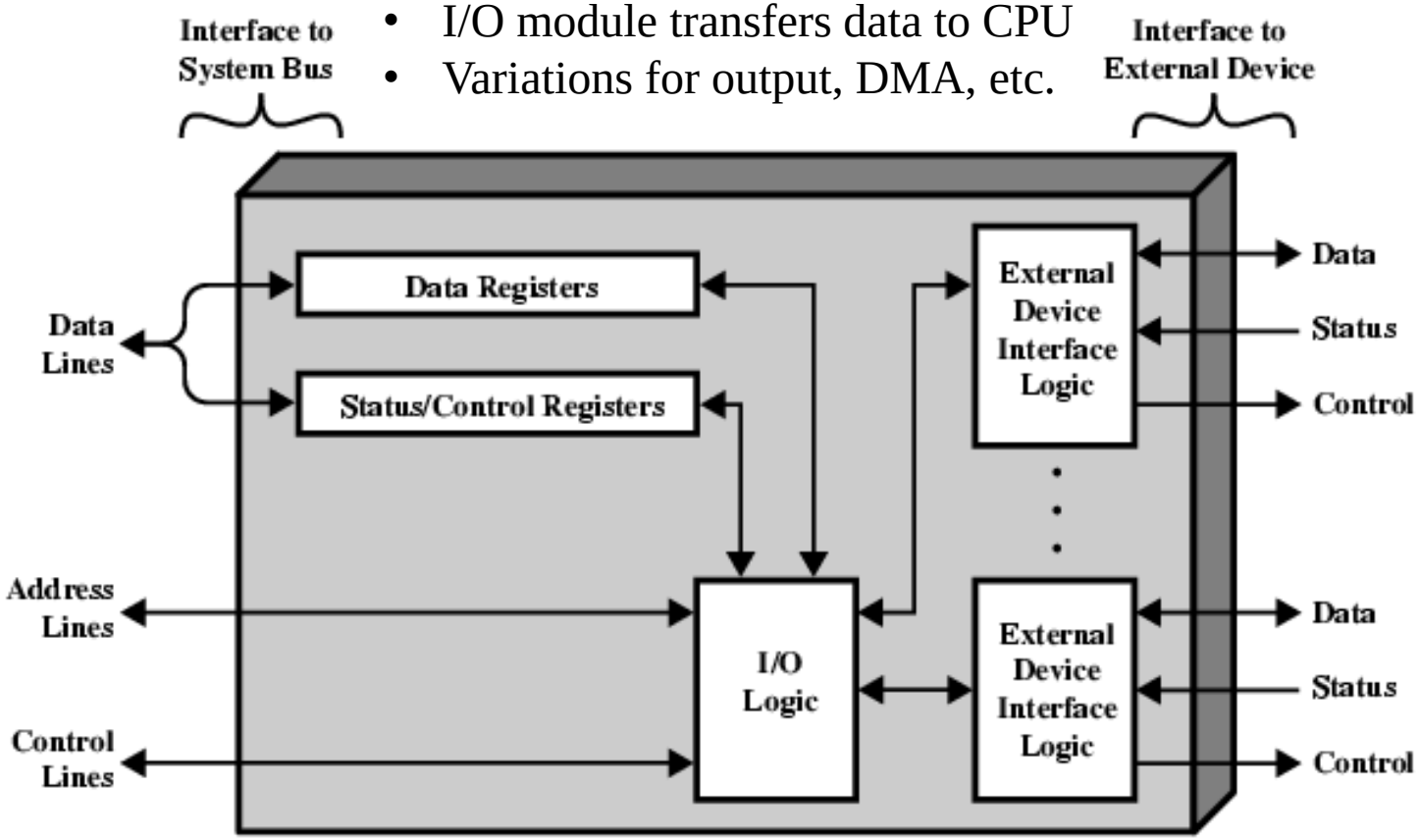
I/O Module Function

- The major functions for an I/O module fall into the following categories:
- Control & Timing
  - Coordinates the flow of traffic between internal resources and external devices
- CPU Communication
  - Involves command decoding, data, status reporting, address recognition
- Device Communication
  - Involves commands, status information, and data
- Data Buffering
  - Performs the needed buffering operation to balance device and memory speeds
- Error Detection: Detects and reports transmission errors

I/O Module Diagram

IO Steps:

- CPU checks I/O module device status
- I/O module returns status
- If ready, CPU requests data transfer
- I/O module gets data from device
- I/O module transfers data to CPU
- Variations for output, DMA, etc.



I/O Module Decisions

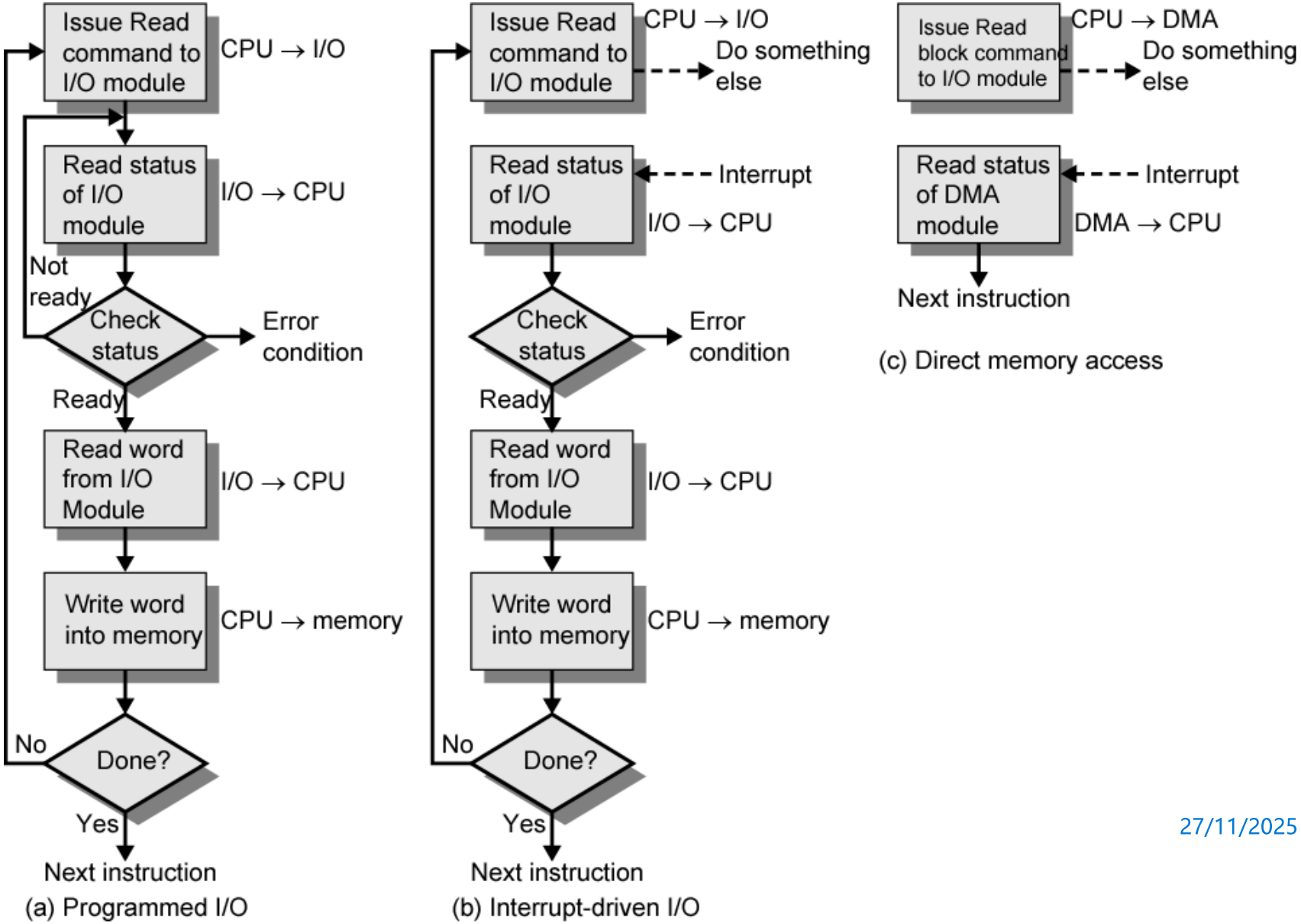
- Hide or reveal device properties to CPU
- Support multiple or single device
- Control device functions or leave for CPU
- Also O/S decisions
  - e.g. Unix treats everything it can as a file

Input Output Techniques

- Programmed
- Interrupt driven
- Direct Memory Access (DMA)

	No Interrupts	**Use of Interrupts**
I/O-to-memory transfer through processor	Programmed I/O	Interrupt-driven I/O
Direct I/O-to-memory transfer		Direct memory access (DMA)

Three Techniques for Input of a Block of Data



Programmed I/O

- CPU has direct control over I/O
  - Sensing status
  - Read/write commands
  - Transferring data
- CPU waits for I/O module to complete operation
- Wastes CPU time

Programmed I/O - detail

- CPU requests I/O operation
- I/O module performs operation
- I/O module sets status bits
- CPU checks status bits periodically
- I/O module does not inform CPU directly
- I/O module does not interrupt CPU
- CPU may wait or come back later

I/O Commands

- CPU issues address
  - Identifies module (& device if >1 per module)

- CPU issues command
  - Control - telling module what to do
    - e.g. spin up disk
  - Test - check status
    - e.g. power? Error?
  - Read/Write
    - Module transfers data via buffer from/to device

Addressing I/O Devices

- Under programmed I/O data transfer is very like memory access (CPU viewpoint)
- Each device given unique identifier
- CPU commands contain identifier (address)

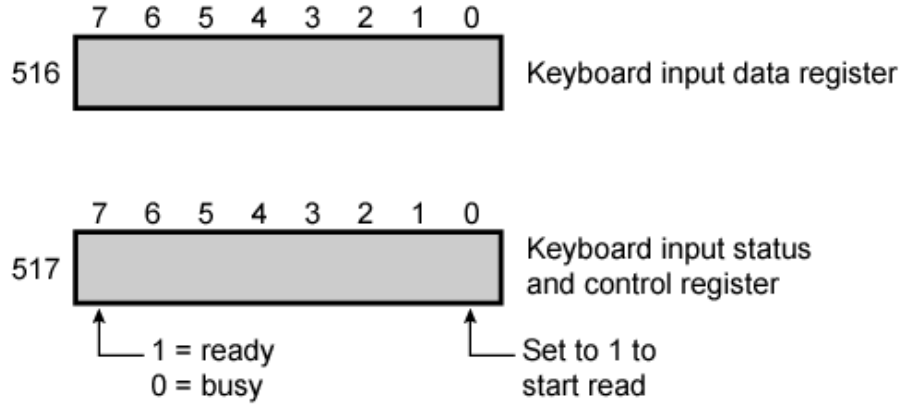
I/O Mapping

When the processor ,main memory and I/O share a common bus two modes of addressing used:

- Memory mapped I/O
  - Devices and memory share an address space
  - I/O looks just like memory read/write
  - No special commands for I/O
    - Large selection of memory access commands available

- Isolated I/O
  - Separate address spaces
  - Need I/O or memory select lines
  - Special commands for I/O
    - Limited set

Memory Mapped and Isolated I/O



ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load AC	"1"	Load accumulator
	Store AC	517	Initiate keyboard read
202	Load AC	517	Get status byte
	Branch if Sign = 0	202	Loop until ready
	Load AC	516	Load data byte

(a) Memory-mapped I/O

ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load I/O	5	Initiate keyboard read
201	Test I/O	5	Check for completion
	Branch Not Ready	201	Loop until complete
	In	5	Load data byte

(b) Isolated I/O

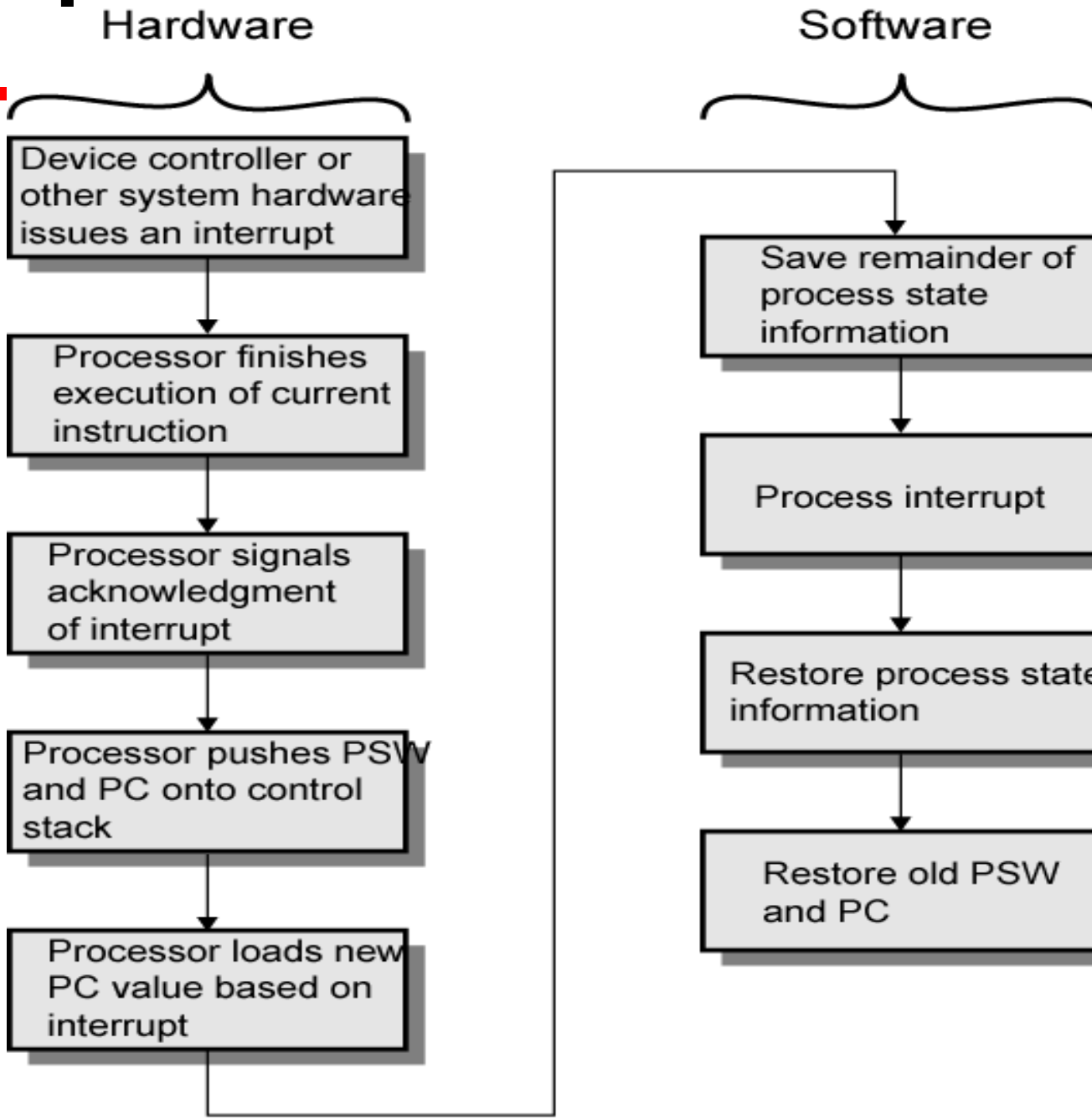
Interrupt Driven I/O

- Overcomes CPU waiting
- No repeated CPU checking of device
- I/O module interrupts when ready

Interrupt Driven I/O
Basic Operation

- CPU issues read command
- I/O module gets data from peripheral whilst CPU does other work
- I/O module interrupts CPU
- CPU requests data
- I/O module transfers data

Simple Interrupt Processing



CPU Viewpoint

- Issue read command
- Do other work
- Check for interrupt at end of each instruction cycle
- If interrupted:-
  - Save context (registers)
  - Process interrupt
    - Fetch data & store
- See Operating Systems notes

Design Issues

- How do you identify the module issuing the interrupt?

- How do you deal with multiple interrupts?
  - i.e. an interrupt handler being interrupted

Identifying Interrupting Module (1)

- Different line for each module
  - PC
  - Limits number of devices

- Software poll
  - CPU asks each module in turn
  - Slow

Identifying Interrupting Module (2)

- Daisy Chain or Hardware poll
  - Interrupt Acknowledge sent down a chain
  - Module responsible places vector on bus
  - CPU uses vector to identify handler routine

- Bus Master
  - Module must claim the bus before it can raise interrupt
  - e.g. PCI & SCSI

Multiple Interrupts

- Each interrupt line has a priority
- Higher priority lines can interrupt lower priority lines
- If bus mastering only current master can interrupt

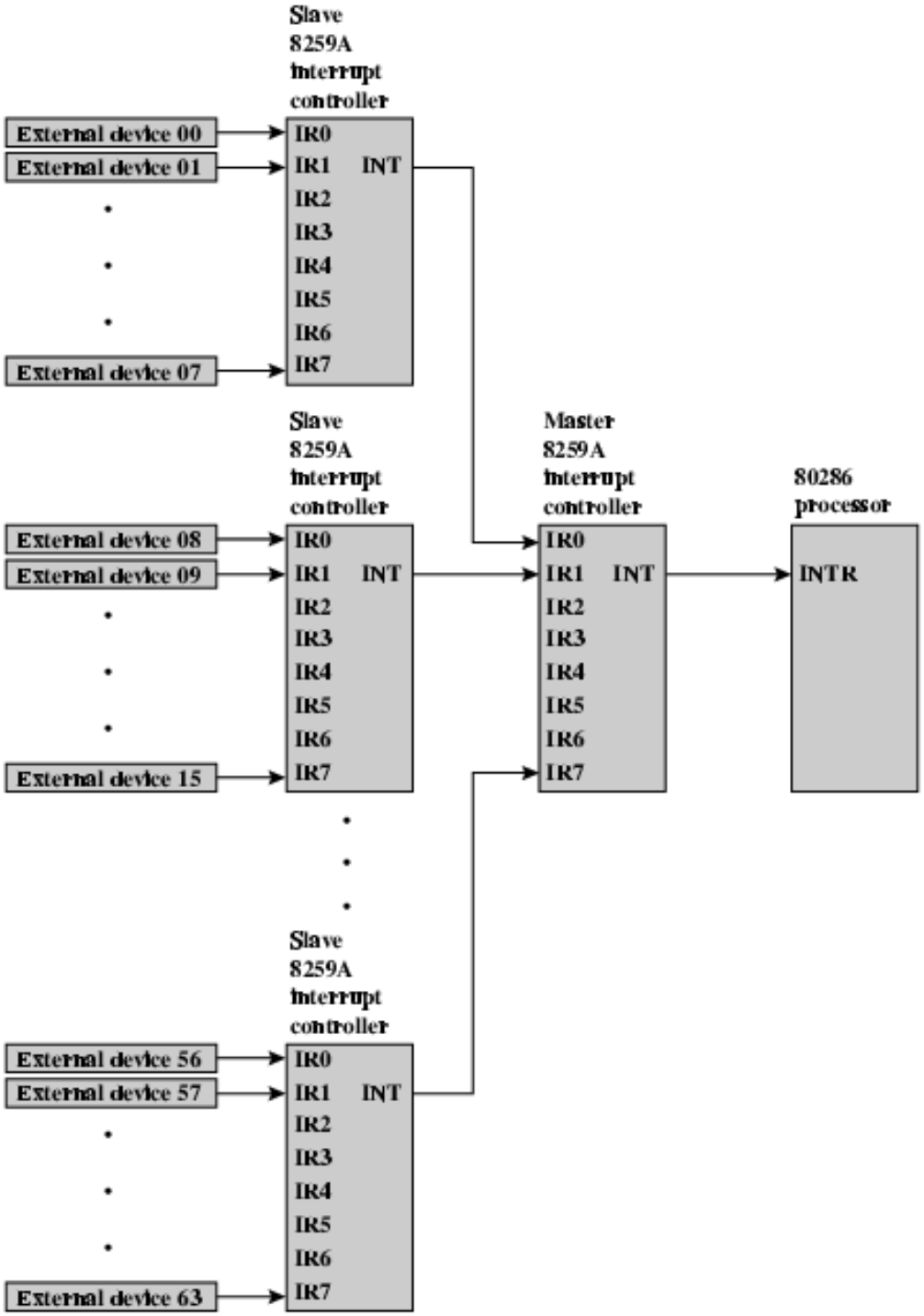
Example - PC Bus

- 80x86 has one interrupt line
- 8086 based systems use one 82C59A interrupt controller
- 82C59A has 8 interrupt lines and the following interrupts modes:
  - Fully Nested: the interrupt requests are ordered in priority from (IR0 through IR7)
  - Rotating: in some application a number of interrupting devices are equal priority. In this mode a device , after being serviced, receives the lowest priority
  - Special mask: this allows the processor to inhibit interrupts from certain devices

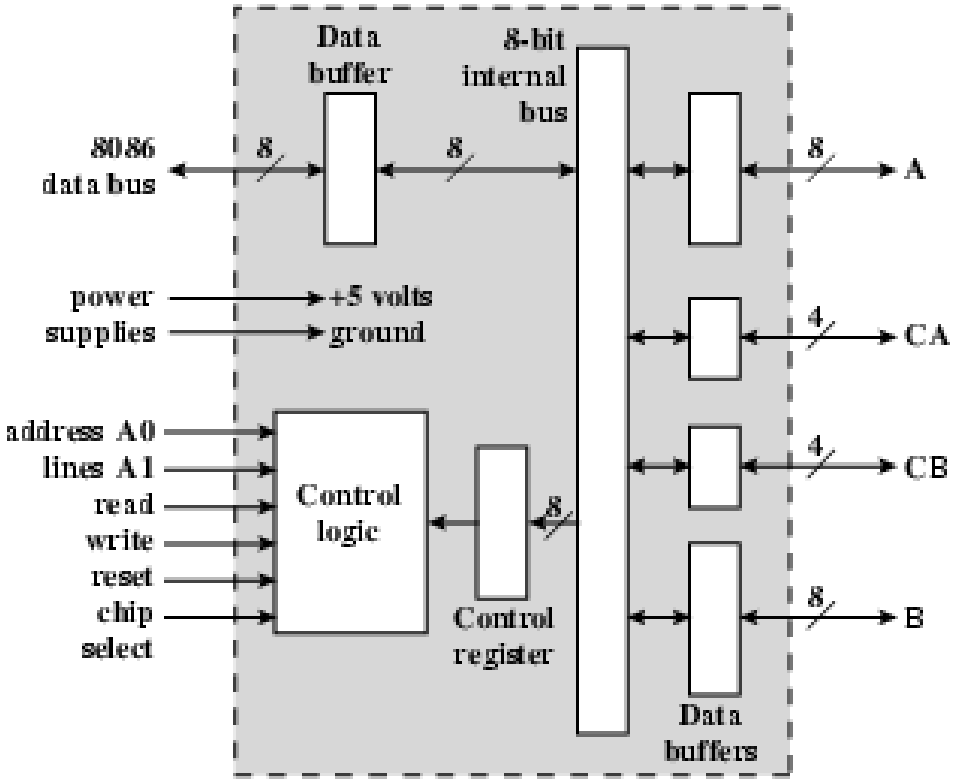
Sequence of Events

- 8259A accepts interrupts
- 8259A determines priority
- 8259A signals 8086 (raises INTR line)
- CPU Acknowledges
- 8259A puts correct vector on data bus
- CPU processes interrupt

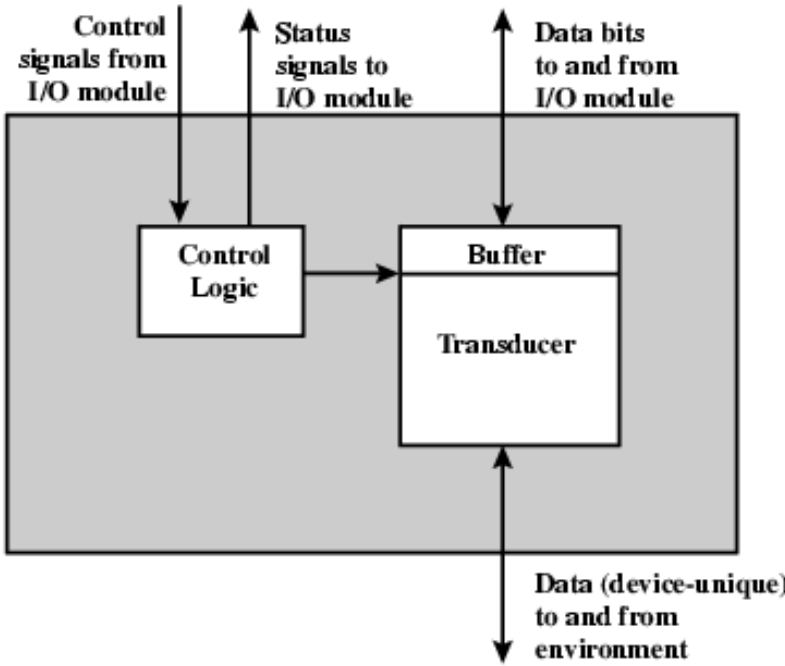
82C59A Interrupt Controller



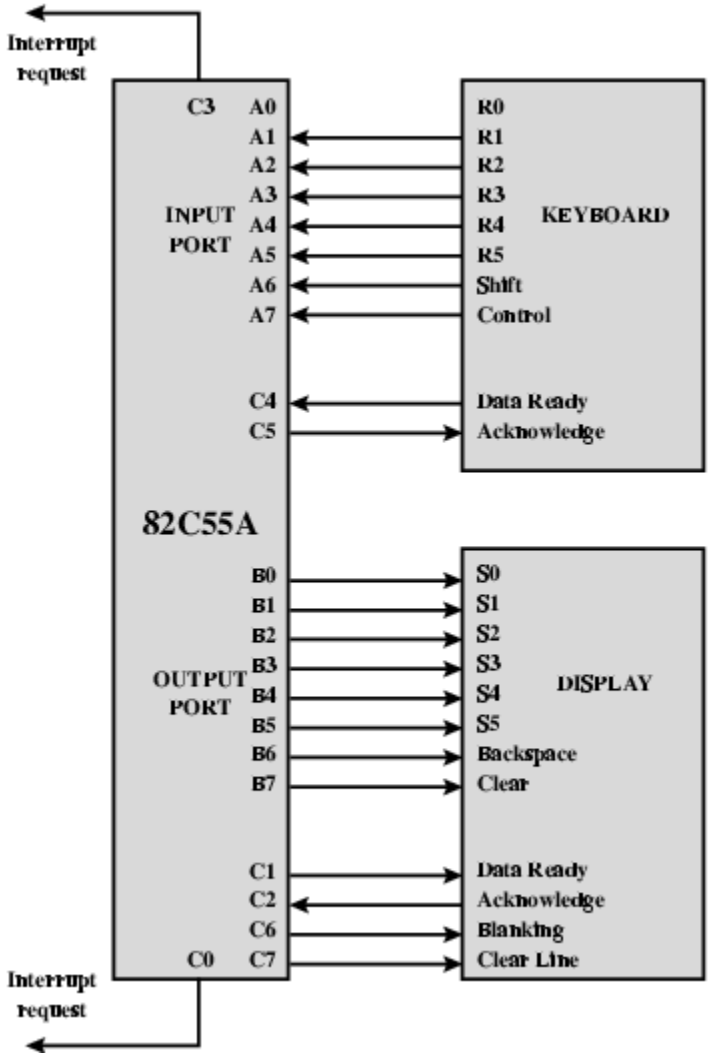
Intel 82C55A Programmable Peripheral Interface



(a) Block diagram



Keyboard/Display Interfaces to 82C55A



Direct Memory Access

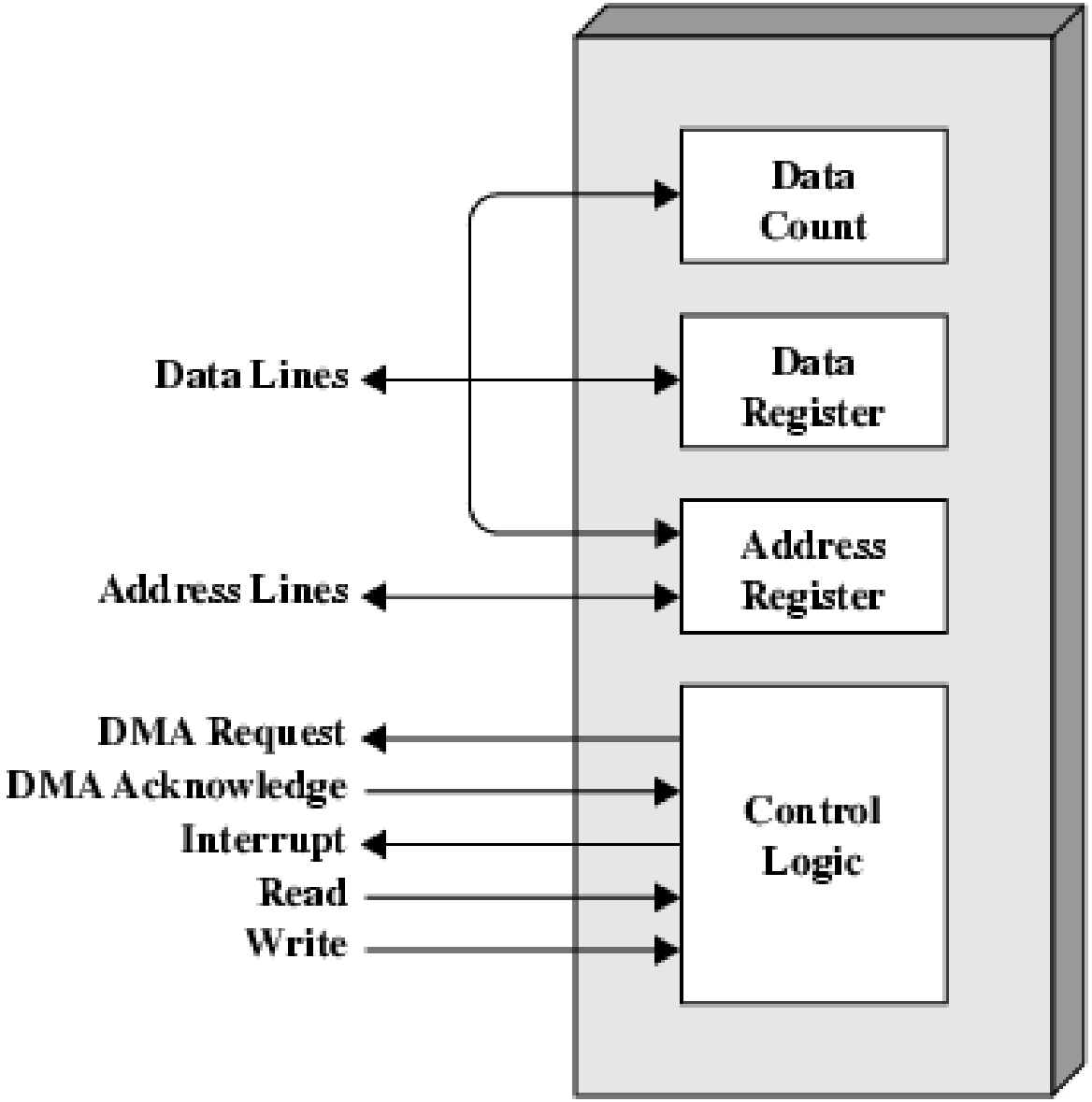
- Interrupt driven and programmed I/O require active CPU intervention
  - Transfer rate is limited
  - CPU is tied up

- DMA is the answer

DMA Function

- Additional Module (hardware) on bus
- DMA controller takes over from CPU for I/O

Typical DMA Module Diagram



DMA Operation

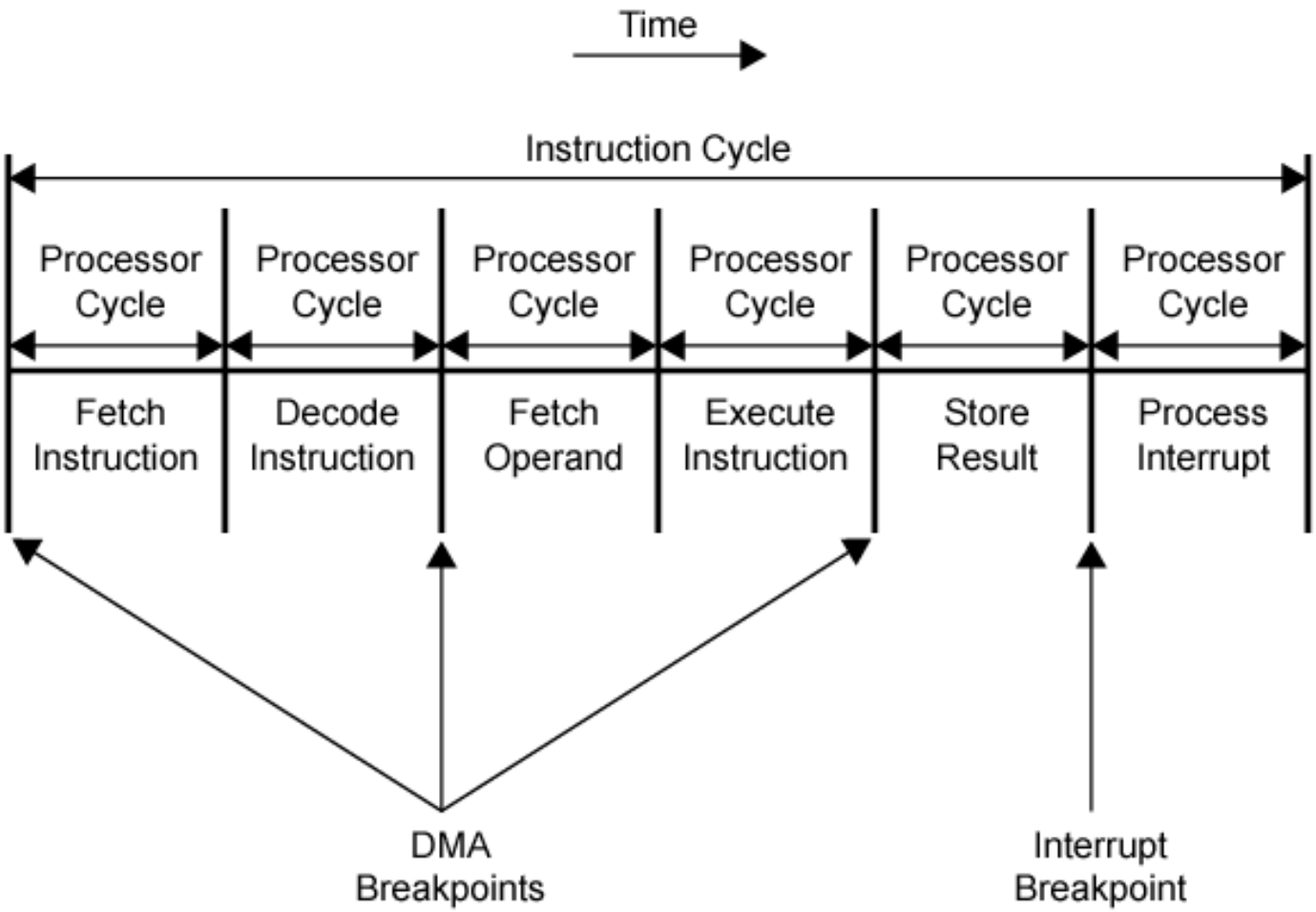
- CPU tells DMA controller:-
  - Read/Write
  - I/O Device address
  - Starting address of memory block for data
  - Amount of data to be transferred
  - The CPU then sends commands to a peripheral device to initiate transfer of data
- CPU carries on with other work
- DMA controller deals with transfer
- DMA controller sends interrupt when finished

DMA Transfer Modes ……

1.Cycle Stealing

- DMA controller takes over bus for a cycle
- Transfer of one word of data
- Not an interrupt
  - CPU does not switch context
- CPU suspended just before it accesses bus
  - i.e. before an operand or data fetch or a data write
- Slows down CPU but not as much as CPU doing transfer
- useful for controllers that monitor data in real time.

DMA and Interrupt Breakpoints During an Instruction Cycle



DMA Transfer Modes…. continue

2. Burst mode(Block Transfer Mode)

Entire block of data is transferred in one contiguous sequence.

Once the DMA controller is granted access to the system bus by the CPU, it transfers all bytes of data in the data block before releasing control of the system buses back to the CPU

Disadvantage

Causes the CPU inactive for relatively long periods of time

DMA Transfer Modes

3.Transparent mode

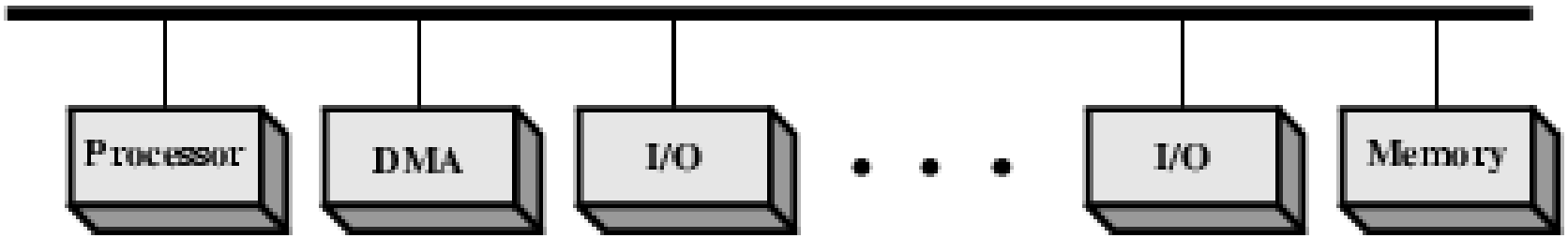
the DMA controller transfers data only when the CPU is performing operations that do not use the system buses.

advantage of transparent mode is that the CPU never stops executing its programs

So it is the most efficient mode in terms of overall system performance

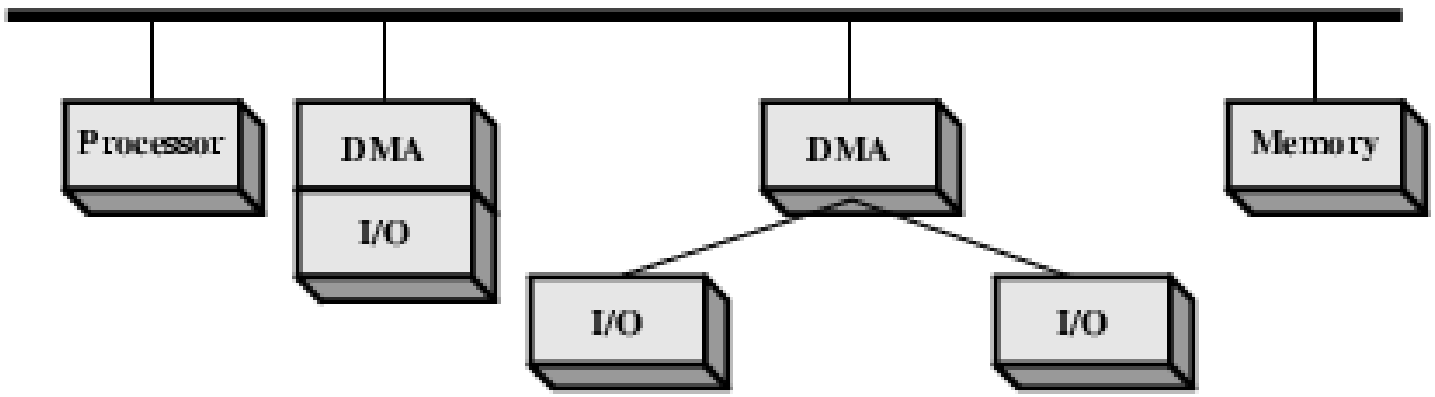
- *disadvantage is that the hardware needs to determine when the CPU is not using the system buses, which can be complex*

DMA Configurations (1)



- Single Bus, Detached DMA controller
- Each transfer uses bus twice
  - I/O to DMA then DMA to memory
- CPU is suspended twice

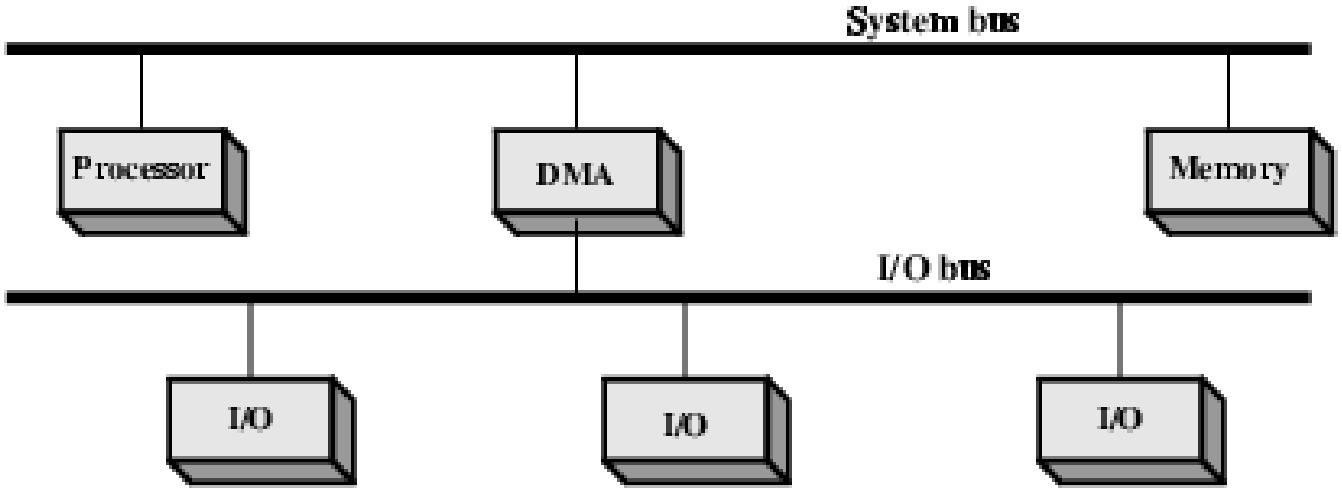
DMA Configurations (2)



(b) Single-bus, Integrated DMA-I/O

- Single Bus, Integrated DMA controller
- Controller may support >1 device
- Each transfer uses bus once
  - DMA to memory
- CPU is suspended once

DMA Configurations (3)



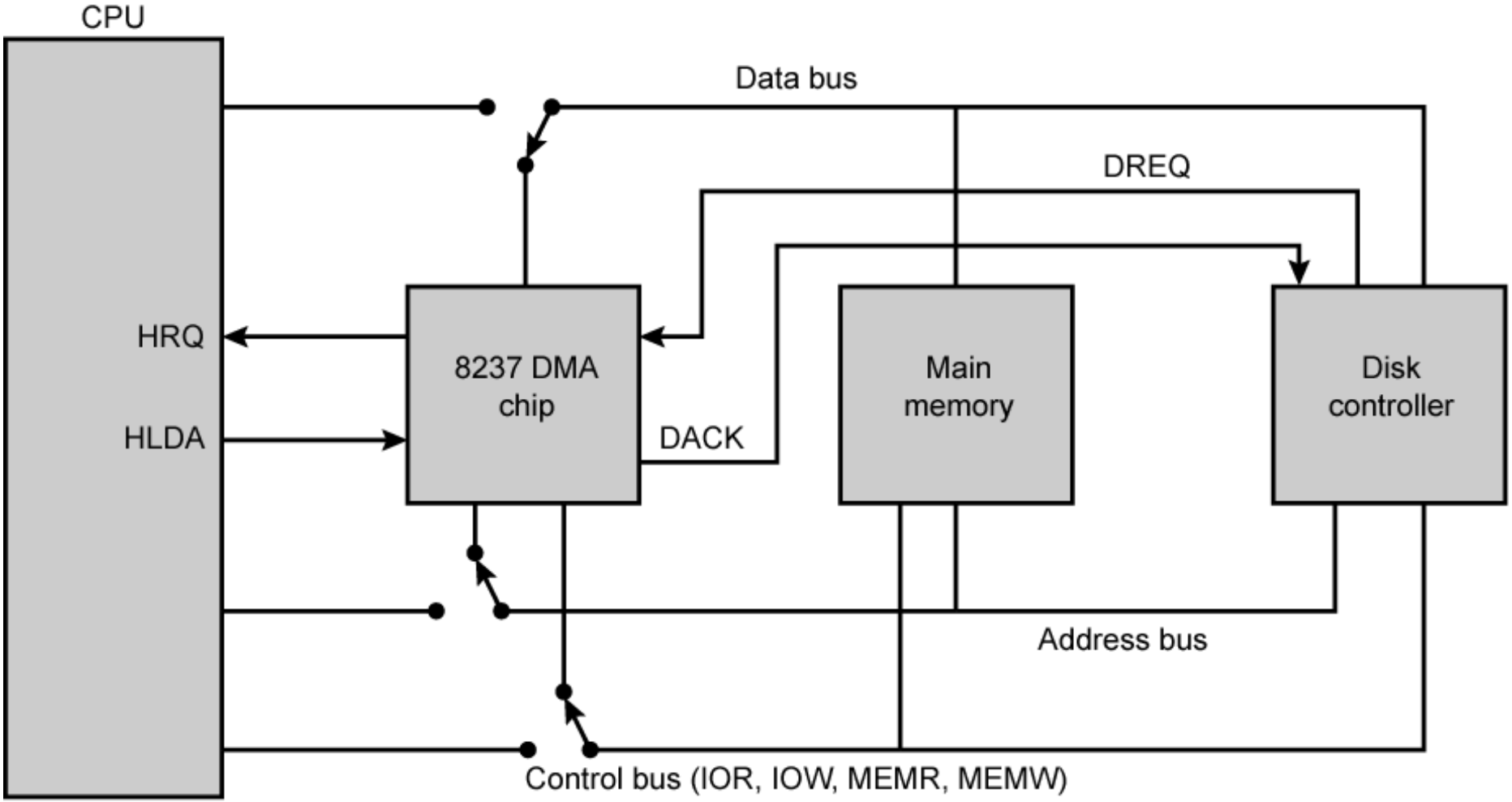
(c) I/O bus

- Separate I/O Bus
- Bus supports all DMA enabled devices
- Each transfer uses bus once
  - DMA to memory
- CPU is suspended once

Intel 8237A DMA Controller

- Interfaces to 80x86 family and DRAM
- When DMA module needs buses it sends HOLD signal to processor
- CPU responds HLDA (hold acknowledge)
  - DMA module can use buses
- E.g. transfer data from memory to disk
  1. Device requests service of DMA by pulling DREQ (DMA request) high
  2. DMA puts high on HRQ (hold request),
  3. CPU finishes present bus cycle (not necessarily present instruction) and puts high on HDLA (hold acknowledge). HOLD remains active for duration of DMA
  4. DMA activates DACK (DMA acknowledge), telling device to start transfer
  5. DMA starts transfer by putting address of first byte on address bus and activating MEMR; it then activates IOW to write to peripheral. DMA decrements counter and increments address pointer. Repeat until count reaches zero
  6. DMA deactivates HRQ, giving bus back to CPU

8237 DMA Usage of Systems Bus



DACK = DMA acknowledge
DREQ = DMA request
HLDA = HOLD acknowledge
HRQ = HOLD request

IOR (I-O Read)
IOW (I-O Write)
MEMR (Memory Write)
MEMW (Memory Write)

Fly-By

- While DMA using buses processor idle
- Processor using bus, DMA idle
  - Known as fly-by DMA controller
- Data does not pass through and is not stored in DMA chip
  - DMA only between I/O port and memory
  - Not between two I/O ports or two memory locations
- Can do memory to memory via register
- 8237 contains four DMA channels
  - Programmed independently
  - Any one active
  - Numbered 0, 1, 2, and 3

I/O Channels

- I/O devices getting more sophisticated
- e.g. 3D graphics cards
- CPU instructs I/O controller to do transfer
- I/O controller does entire transfer
- Improves speed
  - Takes load off CPU
  - Dedicated processor is faster

The Evolution of I/O Function

- The CPU directly controls a peripheral device.
- A controller or I/O module is added. The CPU uses programmed I/O without interrupts.
- Same configuration as in step 2 is used, but now interrupts are employed. The CPU need not spend time waiting for an I/O operation to be performed, thus increasing efficiency.
- The I/O module is given direct access to memory via DMA. It can now move a block of data to or from memory without involving the CPU, except at the beginning and end of the transfer.
- The I/O module is enhanced to become a processor in its own right, with a specialized instruction set tailored for I/O
- The I/O module has a local memory of its own and is, in fact, a computer in its own right. With this architecture a large set of I/O devices can be controlled with minimal CPU involvement.

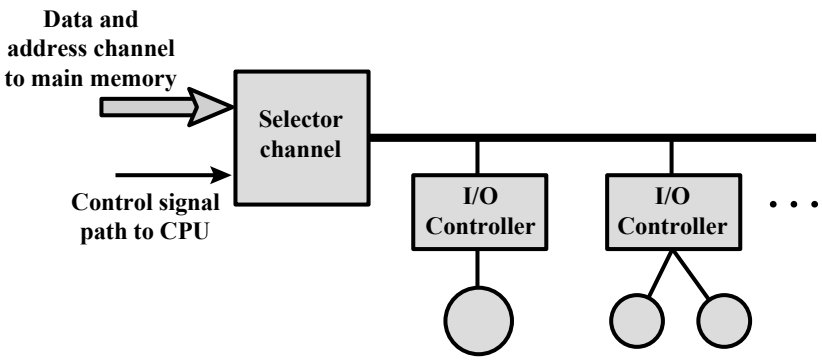
Types of I/O channels

Two types of I/O channels are common:

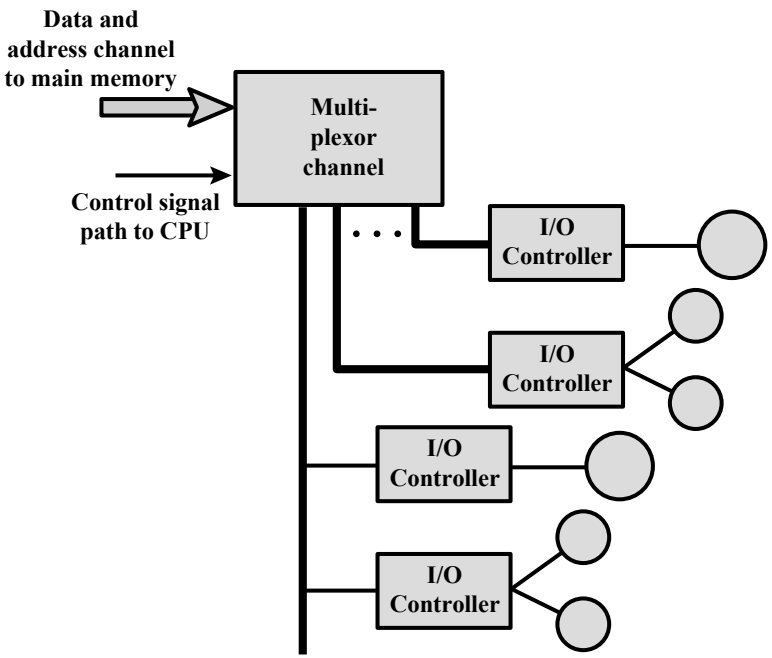
•***selector channel:*** controls multiple high-speed devices and, at any one time, is dedicated to the transfer of data with one of those devices. Thus, the I/O channel selects one device and effects the data transfer. Each device, or a small set of devices, is handled by a *controller*, or I/O module, that is much like the I/O modules we have been discussing. Thus, the I/O channel serves in place of the CPU in controlling these I/O controllers.

•***multiplexor channel****:* can handle I/O with multiple devices at the same time. For low-speed devices, a *byte multiplexor* accepts or transmits characters as fast as possible to multiple devices. For example, the resultant character stream from three devices with different rates and individual streamsA1A2A3A4 . . ., B1B2B3B4 . . ., and C1C2C3C4 . . . might be A1B1C1A2C2A3B2C3A4,and so on. For high-speed devices, a *block multiplexor* interleaves blocks of data from several devices.

I/O Channel Architecture



(a) Selector



(b) Multiplexor

Interfacing

- Connecting devices together
- Bit of wire?
- Dedicated processor/memory/buses?
- E.g. FireWire, InfiniBand

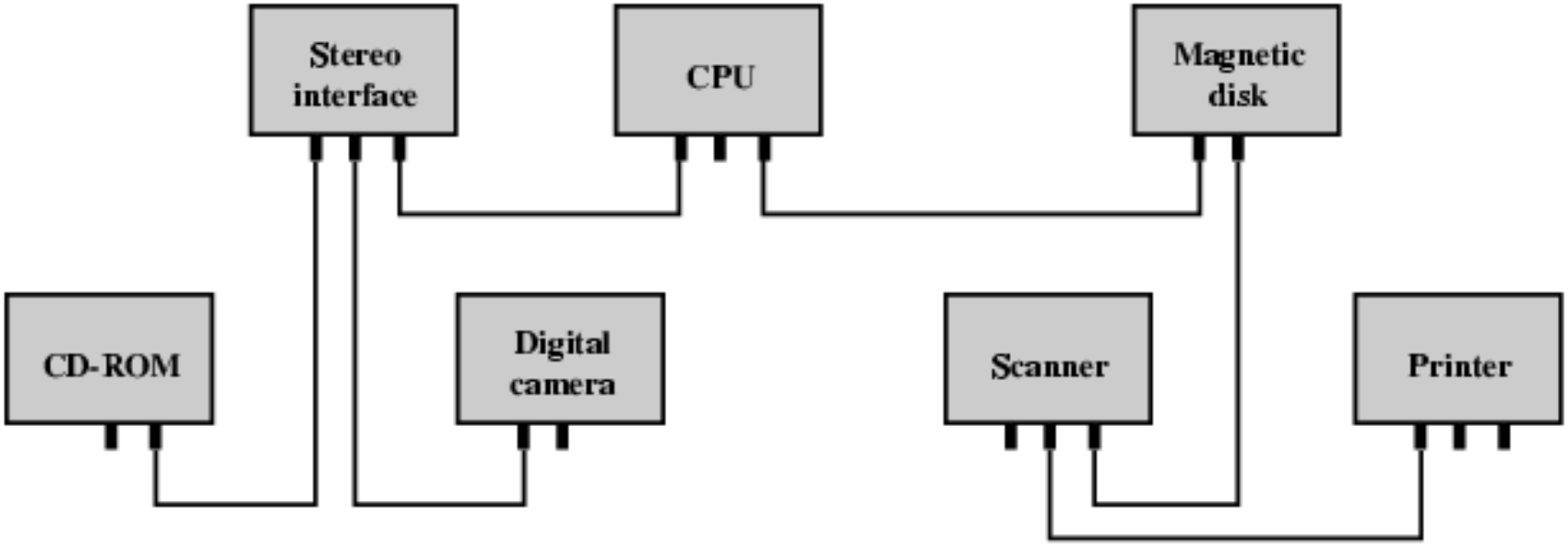
IEEE 1394 FireWire

- High performance serial bus
- Fast
- Low cost
- Easy to implement
- Also being used in digital cameras, VCRs and TV

FireWire Configuration

- Daisy chain
- Up to 63 devices on single port
  - Really 64 of which one is the interface itself
- Up to 1022 buses can be connected with bridges
- Automatic configuration
- No bus terminators
- May be tree structure

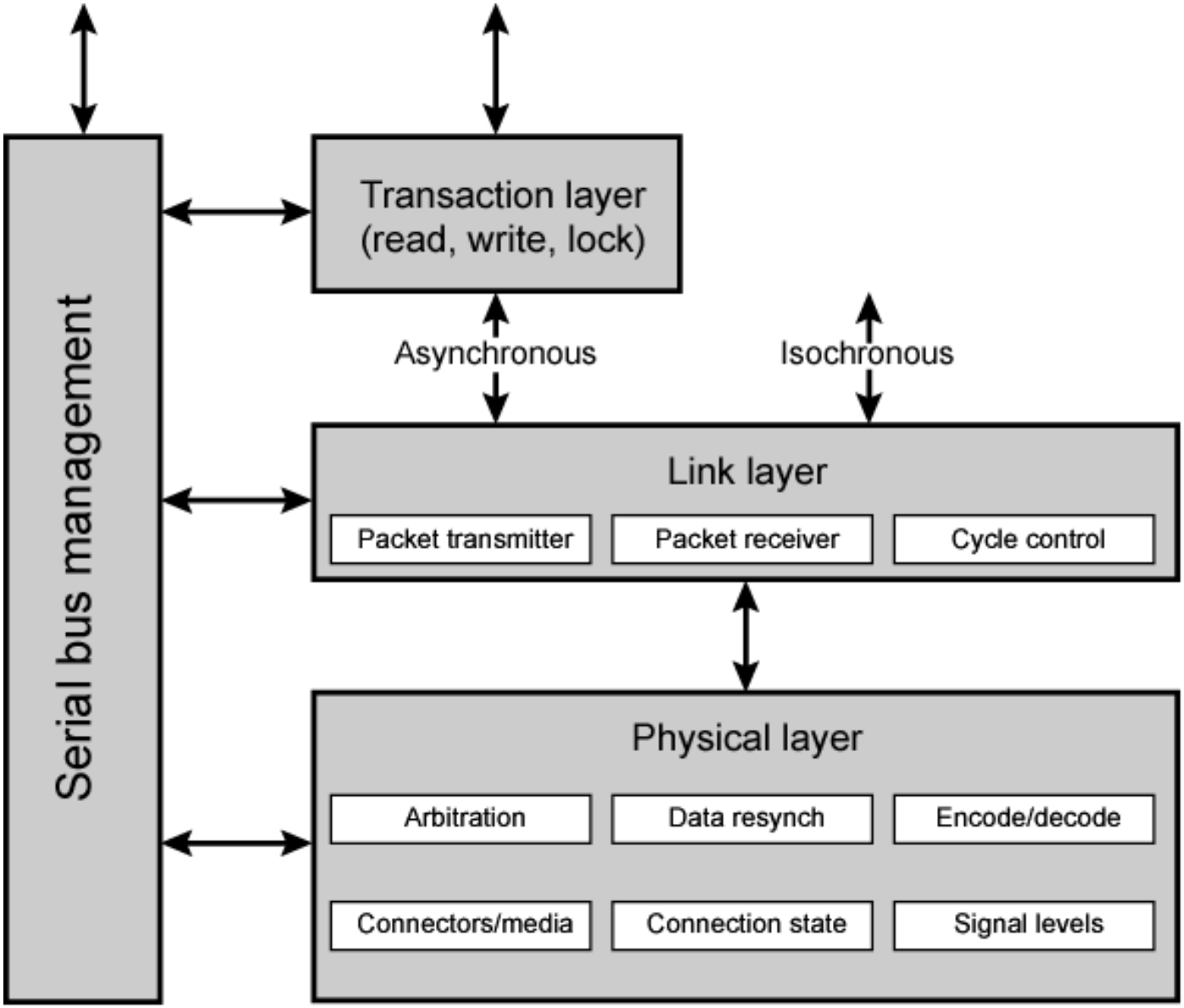
Simple FireWire Configuration



FireWire 3 Layer Stack

- Physical
  - -Defines the Transmission medium, electrical and signaling characteristics
  - -Provides arbitration services
- Link
  - — Describes the transmission of data in packets
- Transaction
  - — Defines Request-response protocol that hides the lower layer details of firewire from Applications

FireWire Protocol Stack



FireWire - Link Layer

- Two transmission types
  - Asynchronous
    - Variable amount of data and several bytes of transaction layer information transferred as a packet
    - To an explicit address
    - Acknowledgement returned

    Used by data that have no fixed data rate requirements
  - Isochronous
    - Variable amount of data is transferred in a sequence of fixed size packets at regular intervals
    - Simplified addressing
    - No acknowledgement

    Used for devices that regularly generate or consume data such as digital sound or video

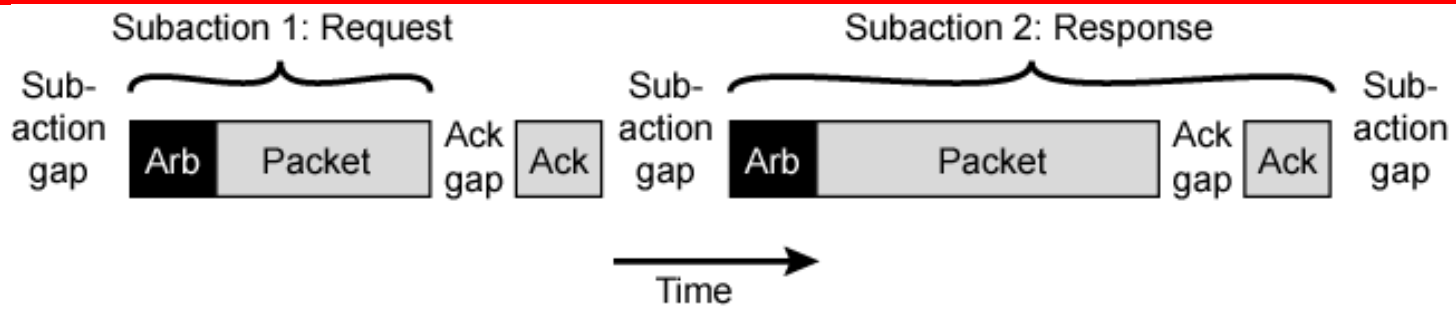
Mixed traffic load of isochronous and asynchronous accommodated as following:

- One node is designated as cycle master
- Periodically the cycle master issues a cycle_start packet this signals all other nodes that an isochronous cycle has been started during this cycle only isochronous packets may be sent.
- Each isochronous data source arbitrates for bus access
- The winning node immediately transmits a packet
- no acknowledgment a small gap between the transmission of one packet and the arbitration period for the next packet
- After all isochronous sources have transmitted the bus will remain idle long enough for subaction gap to occur- this is the signal to the asynchronous sources to start competition for buss access

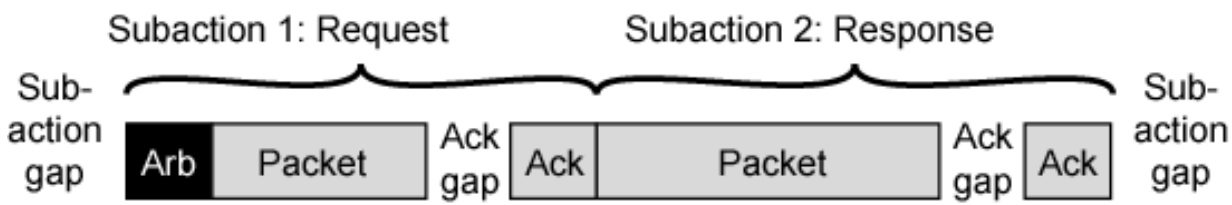
The process of delivering a single packet is called subaction and it is consists of five time periods

- Arbitration sequence: this is the exchange of signals required to give one device control of the bus
- Packet transmission every packet include a header containing source, destination ids, packet type information ,CRC, user data.
- Acknowledgment gap: Time delay for destination to receive and decode packet and generate an acknowledgment.
- Acknowledgment: the recipient of the packet return an acknowledgment packet with a code indicating the action taken by recipient.
- Subaction gap: This is an enforced idle period to ensure that other nodes on the bus do not begin arbitration before acknowledgment packet has been transmitted.

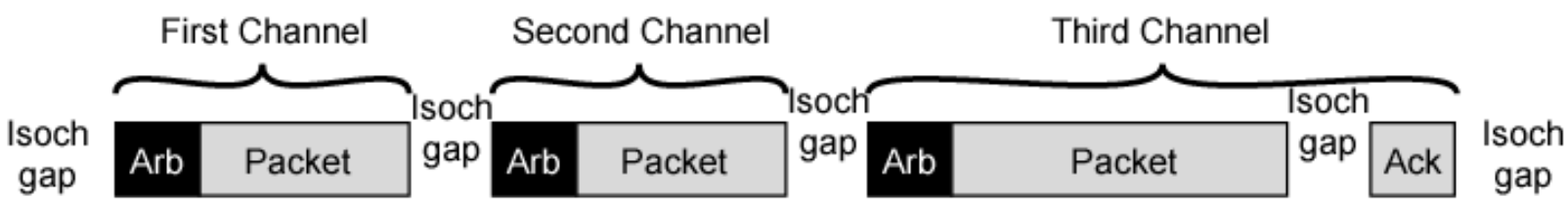
FireWire Subactions (process of delivering a single packet)



(a) Example asynchronous subaction



(b) Concatenated asynchronous subactions



(c) Example isochronous subactions

InfiniBand

- I/O specification aimed at high end servers
  - Merger of Future I/O (Cisco, HP, Compaq, IBM) and Next Generation I/O (Intel)
- Version 1 released early 2001
- The Standard describe Architecture and specification for data flow between processor and intelligent I/O devices
- Intended to replace PCI in servers
- Increased capacity, expandability, flexibility

InfiniBand Architecture

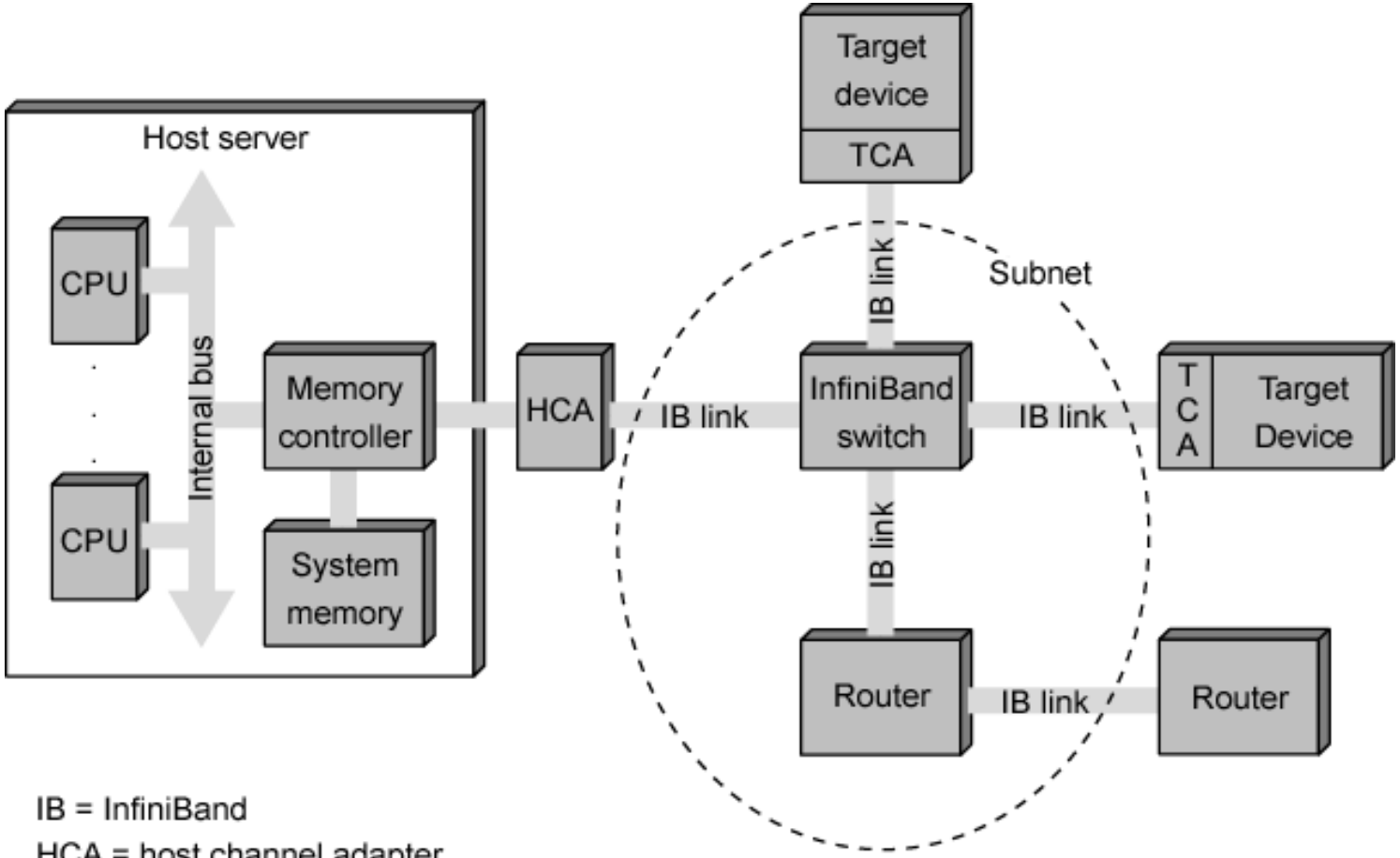
- Remote storage, networking and connection between servers
- Attach servers, remote storage, network devices to central fabric of switches and links

Removing I/O from Server Chassis allows :

  - Greater server density
  - Scalable data centre

- Independent nodes added as required up to 64,000 nodes
- I/O distance from server up to
  - 17m using copper
  - 300m multimode fibre optic
  - 10km single mode fibre
- Up to 30Gbps

InfiniBand Switch Fabric

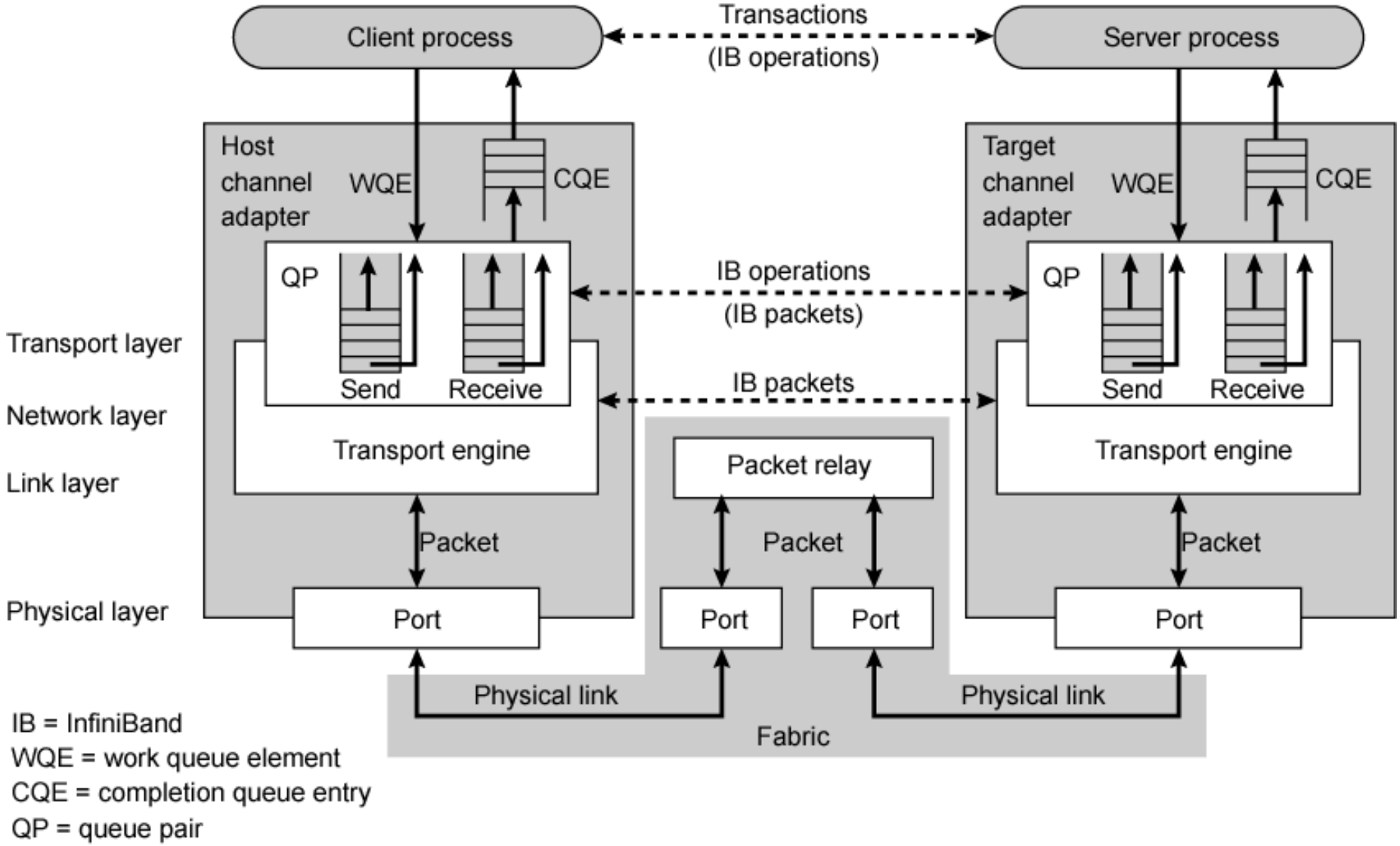


IB = InfiniBand
HCA = host channel adapter
TCA = target channel adapter

InfiniBand Operation

- 16 logical channels (virtual lanes) per physical link
- One lane for management, rest for data
- Data in stream of packets
- Virtual lane dedicated temporarily to end to end transfer
- Switch maps traffic from incoming to outgoing lane

InfiniBand Protocol Stack



William Stallings
Computer Organization and Architecture
7th Edition

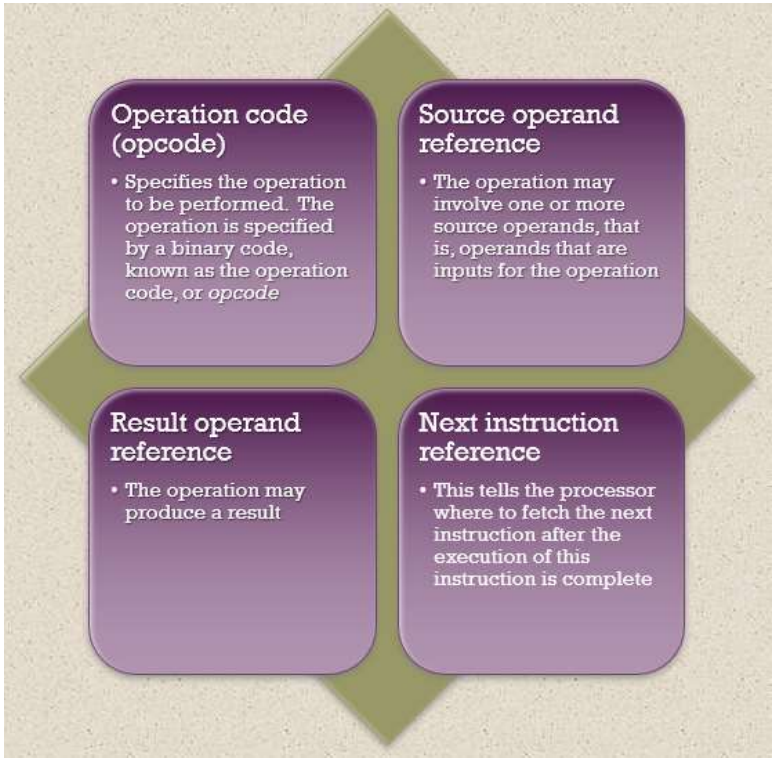
Instruction Sets:
Characteristics and Functions

What is an Instruction Set?

- Instruction set: the complete collection of instructions that are understood and executed by a CPU
- The operation of the processor is determined by the instructions it executes, referred to as machine instructions or computer instructions
- Machine Code: Binary
- Usually represented by assembly codes
- Each instruction must contain the information required by the processor for execution

Elements of an Instruction

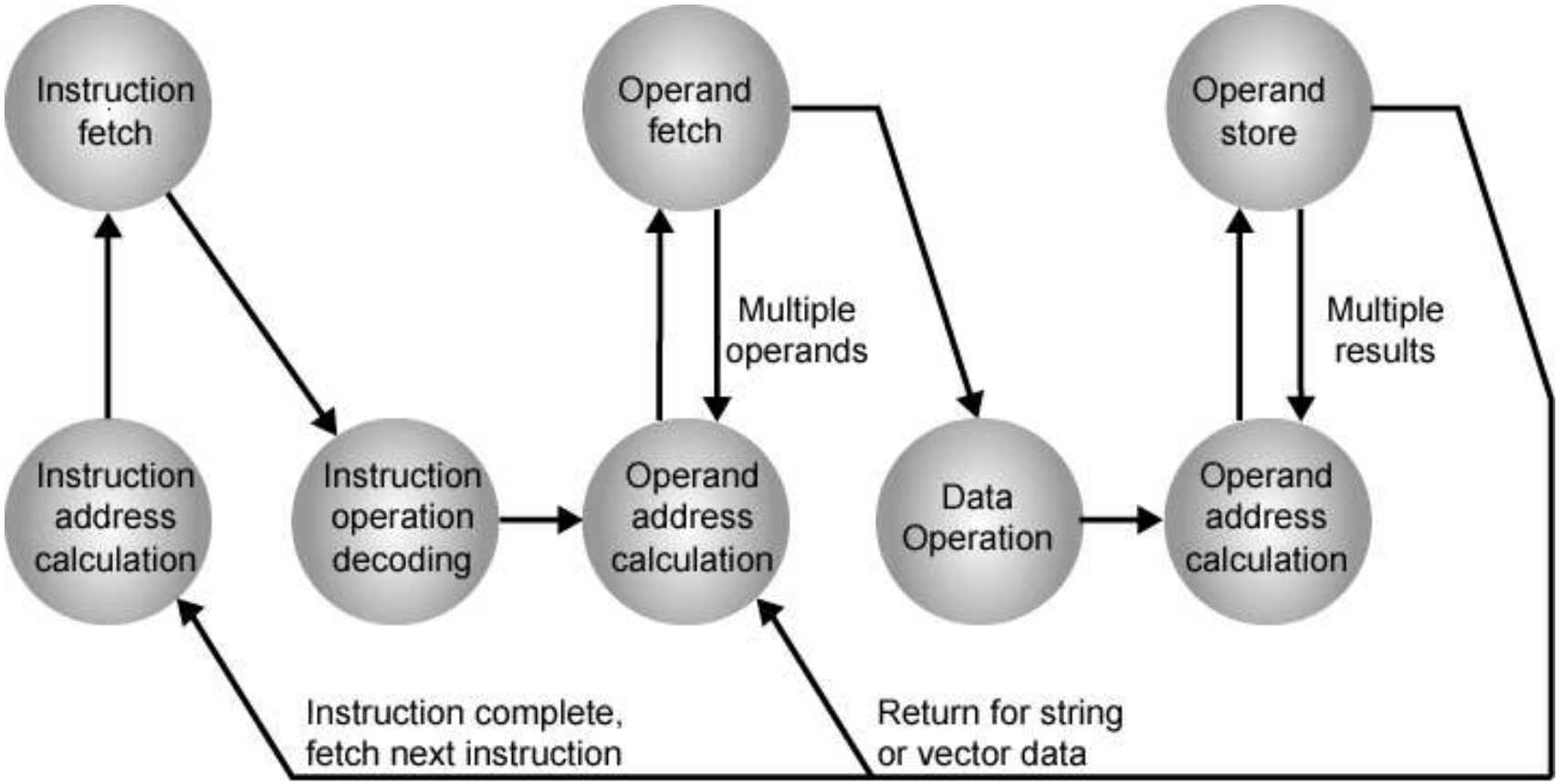
- Operation code (Op code)
  - Do this (operation)
- Source Operand reference
  - To this (input)
- Result Operand reference
  - Put the answer here (outpu)
- Next Instruction Reference
  - When you have done that, do this...



Where have all the Operands Gone?

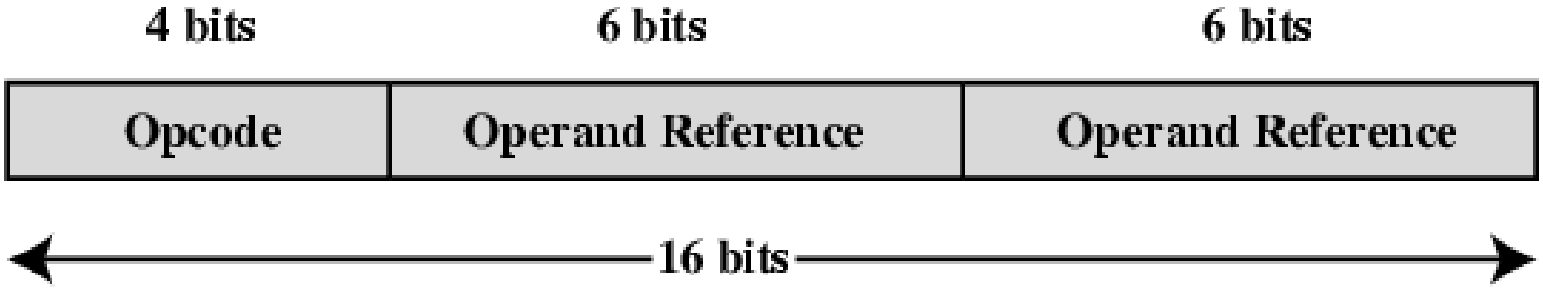
- Main or virtual memory
  - As with next instruction references, the main or virtual memory address must be supplied
- I/O device
  - The instruction must specify the I/O module and device for the operation. If memory-mapped I/O is used, this is just another main or virtual memory address
- Processor register
  - A processor contains one or more registers that may be referenced by machine instructions.
  - If more than one register exists each register is assigned a unique name or number and the instruction must contain the number of the desired register
- Immediate
  - The value of the operand is contained in a field in the instruction being executed

Instruction Cycle State Diagram



Instruction Format

- In machine code each instruction has a unique bit pattern
- For human consumption (well, programmers anyway) a symbolic representation is used
  - e.g. ADD, SUB, LOAD
- Operands can also be represented in this way
  - ADD A,B



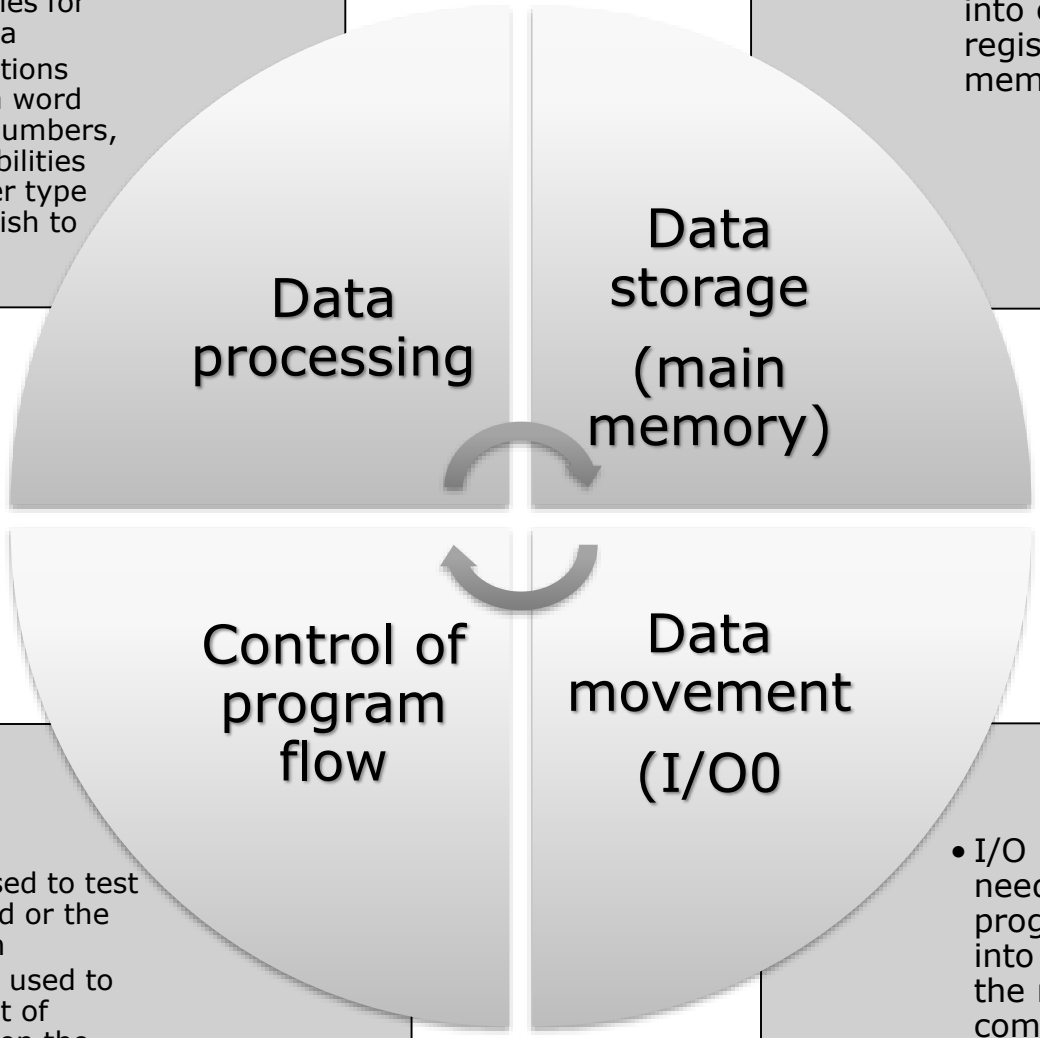
Instruction Representation

- Opcodes are represented by abbreviations called mnemonics
- Examples include:
  - ADD Add
  - SUB Subtract
  - MUL Multiply
  - DIV Divide
  - LOAD Load data from memory
  - STOR Store data to memory
- Operands are also represented symbolically
- Each symbolic opcode has a fixed binary representation
  - The programmer specifies the location of each symbolic operand

Instruction Types

- Arithmetic instructions provide computational capabilities for processing numeric data
- Logic (Boolean) instructions operate on the bits of a word as bits rather than as numbers, thus they provide capabilities for processing any other type of data the user may wish to employ

- Movement of data into or out of register and or memory locations



- Test instructions are used to test the value of a data word or the status of a computation
- Branch instructions are used to branch to a different set of instructions depending on the decision made

- I/O instructions are needed to transfer programs and data into memory and the results of computations back out to the user

Number of Addresses (a)

- 3 addresses
  - Operand 1, Operand 2, Result
  - a = b + c;
  - May be a forth - next instruction (usually implicit)
  - Not common
  - Needs very long words to hold everything

Instruction		Comment
SUB	Y, A, B	$Y \leftarrow A - B$
MPY	T, D, E	$T \leftarrow D \times E$
ADD	T, T, C	$T \leftarrow T + C$
DIV	Y, Y, T	$Y \leftarrow Y \div T$

$$Y = \frac{A - B}{C + (D \times E)}$$

Number of Addresses (b)

- 2 addresses
  - One address doubles as operand and result
  - a = a + b
  - Reduces length of instruction
  - Requires some extra work
    - Temporary storage to hold some results

Instruction	Comment
MOVE Y, A	$Y \leftarrow A$
SUB Y, B	$Y \leftarrow Y - B$
MOVE T, D	$T \leftarrow D$
MPY T, E	$T \leftarrow T \times E$
ADD T, C	$T \leftarrow T + C$
DIV Y, T	$Y \leftarrow Y \div T$

$$Y = \frac{A - B}{C + (D \times E)}$$

Number of Addresses (c)

- 1 address
  - Implicit second address
  - Usually a register (accumulator)
  - Common on early machines

Instruction	Comment
LOAD D	$AC \leftarrow D$
MPY E	$AC \leftarrow AC \times E$
ADD C	$AC \leftarrow AC + C$
STOR Y	$Y \leftarrow AC$
LOAD A	$AC \leftarrow A$
SUB B	$AC \leftarrow AC - B$
DIV Y	$AC \leftarrow AC \div Y$
STOR Y	$Y \leftarrow AC$

$$\mathbf{Y} = \frac{\mathbf{A} - \mathbf{B}}{\mathbf{C} + (\mathbf{D} \times \mathbf{E})}$$

Number of Addresses (d)

- 0 (zero) addresses
  - All addresses implicit
  - Uses a stack
  - e.g. push a
  - push b
  - add
  - pop c
  - c = a + b

AC = accumulator
T = top of stack
(T – 1) = second element of stack
A, B, C = memory or register locations

Number of Addresses	Symbolic Representation	Interpretation
3	OP A, B, C	A ¬ B OP C
2	OP A, B	A ¬ A OP B
1	OP A	AC ¬ AC OP A
0	OP	T ¬ (T – 1) OP T

How Many Addresses

- More addresses
  - More complex (powerful?) instructions
  - More registers
    - Inter-register operations are quicker
  - Fewer instructions per program
- Fewer addresses
  - Less complex (powerful?) instructions
  - More instructions per program
  - Faster fetch/execution of instructions

Design Decisions (1)

- Operation repertoire
  - How many ops?
  - What can they do?
  - How complex are they?
- Data types
- Instruction formats
  - Length of op code field
  - Number of addresses
- Registers
  - Number of CPU registers available
  - Which operations can be performed on which registers?
- Addressing modes (later…)

- RISC v CISC

Design Decisions (2)

Very complex because it affects so many aspects of the computer system

Defines many of the functions performed by the processor

Programmer's means of controlling the processor

Fundamental design issues:

Operation repertoire	Data types	Instruction format	Registers	Addressing
•How many and which operations to provide and how complex operations should be	•The various types of data upon which operations are performed	•Instruction length in bits, number of addresses, size of various fields, etc.	•Number of processor registers that can be referenced by instructions and their use	•The mode or modes by which the address of an operand is specified

Types of Operand

- Addresses
- Numbers
  - Integer/floating point
- Characters
  - ASCII etc.
- Logical Data
  - Bits or flags
- (Aside: Is there any difference between numbers and characters? Ask a C programmer!)

Pentium Data Types

- 8 bit Byte
- 16 bit word
- 32 bit double word
- 64 bit quad word
- Addressing is by 8 bit unit
- A 32 bit double word is read at addresses divisible by 4

Specific Data Types

- General - arbitrary binary contents
- Integer - single binary value
- Ordinal - unsigned integer
- Unpacked BCD (binary coded decimal) - One digit per byte
- Packed BCD - 2 BCD digits per byte
- Near Pointer - 32 bit offset within segment
- Bit field
- Byte String
- Floating Point

PowerPC Data Types

- 8 (byte), 16 (halfword), 32 (word) and 64 (doubleword) length data types
- Some instructions need operand aligned on 32 bit boundary
- Can be big- or little-endian
- Fixed point processor recognises:
  - Unsigned byte, unsigned halfword, signed halfword, unsigned word, signed word, unsigned doubleword, byte string (<128 bytes)
- Floating point
  - IEEE 754
  - Single or double precision

x86 Data Types

Data Type	Description
General	Byte, word (16 bits), doubleword (32 bits), quadword (64 bits), and double quadword (128 bits) locations with arbitrary binary contents.
Integer	A signed binary value co4ntained in a byte, word, or doubleword, using twos complement representation.
Ordinal	An unsigned integer contained in a byte, word, or doubleword.
Unpacked binary coded decimal (BCD)	A representation of a BCD digit in the range 0 through 9, with one digit in each byte.
Packed BCD	Packed byte representation of two BCD digits; value in the range 0 to 99.
Near pointer	A 16-bit, 32-bit, or 64-bit effective address that represents the offset within a segment. Used for all pointers in a nonsegmented memory and for references within a segment in a segmented memory.
Far pointer	A logical address consisting of a 16-bit segment selector and an offset of 16, 32, or 64 bits. Far pointers are used for memory references in a segmented memory model where the identity of a segment being accessed must be specified explicitly.
Bit field	A contiguous sequence of bits in which the position of each bit is considered as an independent unit. A bit string can begin at any bit position of any byte and can contain up to 32 bits.
Bit string	A contiguous sequence of bits, containing from zero to $2_{32} - 1$ bits.
Byte string	A contiguous sequence of bytes, words, or doublewords, containing from zero to $2_{32} - 1$ bytes.
Floating point	See Figure 12.4.
Packed SIMD (single instruction, multiple data)	Packed 64-bit and 128-bit data types

ARM Data Types

All three data types can also be used for twos complement signed integers

ARM processors support data types of:

- 8 (byte)
- 16 (halfword)
- 32 (word) bits in length

Alignment checking

- When the appropriate control bit is set, a data abort signal indicates an alignment fault for attempting unaligned access

For all three data types an unsigned interpretation is supported in which the value represents an unsigned, nonnegative integer

Unaligned access

- When this option is enabled, the processor uses one or more memory accesses to generate the required transfer of adjacent bytes transparently to the programmer

Types of Operation

Data Transfer	Transfer data from one location to another
	If memory is involved: Determine memory address Perform virtual-to-actual-memory address transformation Check cache Initiate memory read/write
Arithmetic	May involve data transfer, before and/or after
	Perform function in ALU
	Set condition codes and flags
Logical	Same as arithmetic
Conversion	Similar to arithmetic and logical. May involve special logic to perform conversion
Transfer of Control	Update program counter. For subroutine call/return, manage parameter passing and linkage
I/O	Issue command to I/O module
	If memory-mapped I/O, determine memory-mapped address

Data Transfer

Most fundamental type of machine instruction

Must specify:

- Location of the source and destination operands
- The length of data to be transferred must be indicated
- The mode of addressing for each operand must be specified

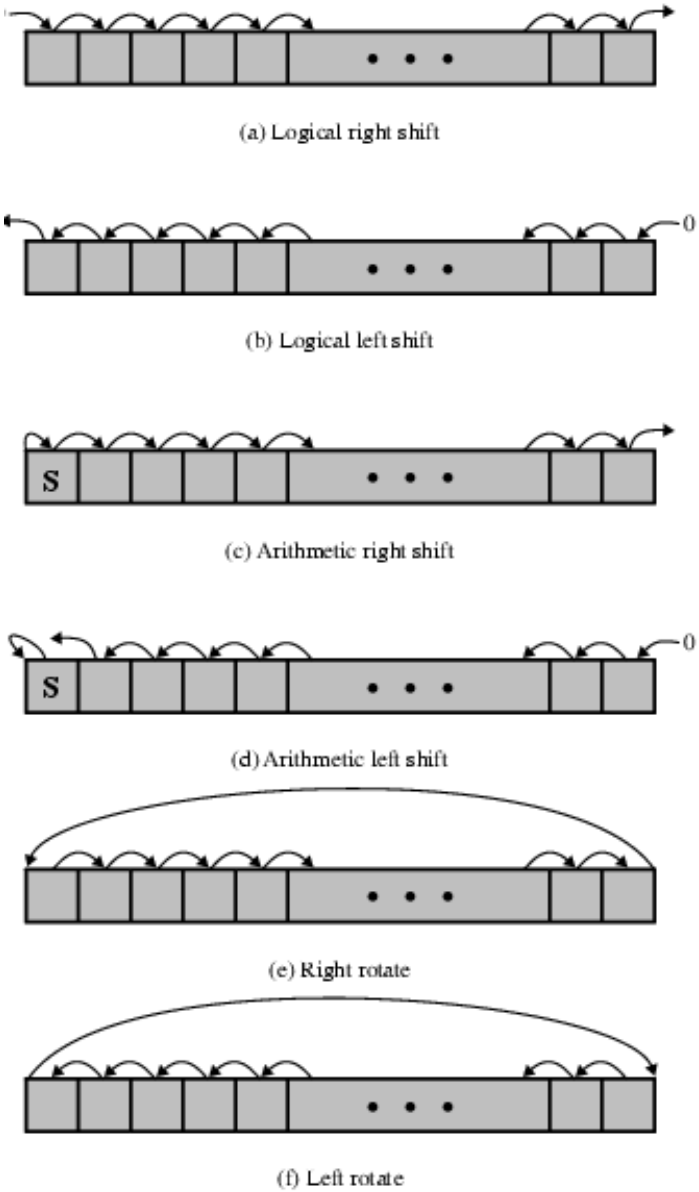
Examples of IBM EAS/390 Data Transfer Operations

Operation Mnemonic	Name	Number of Bits Transferred	Description
L	Load	32	Transfer from memory to register
LH	Load Halfword	16	Transfer from memory to register
LR	Load	32	Transfer from register to register
LER	Load (Short)	32	Transfer from floating-point register to floating-point register
LE	Load (Short)	32	Transfer from memory to floating-point register
LDR	Load (Long)	64	Transfer from floating-point register to floating-point register
LD	Load (Long)	64	Transfer from memory to floating-point register
ST	Store	32	Transfer from register to memory
STH	Store Halfword	16	Transfer from register to memory
STC	Store Character	8	Transfer from register to memory
STE	Store (Short)	32	Transfer from floating-point register to memory
STD	Store (Long)	64	Transfer from floating-point register to memory

Arithmetic

- Add, Subtract, Multiply, Divide
- Signed Integer
- Floating point ?
- May include
  - Absolute (||)
  - Increment (a++)
  - Decrement (a--)
  - Negate (-a)

Shift and Rotate Operations



Input	Operation	Result
10100110	Logical right shift (3 bits)	00010100
10100110	Logical left shift (3 bits)	00110000
10100110	Arithmetic right shift (3 bits)	11110100
10100110	Arithmetic left shift (3 bits)	10110000
10100110	Right rotate (3 bits)	11010100
10100110	Left rotate (3 bits)	00110101

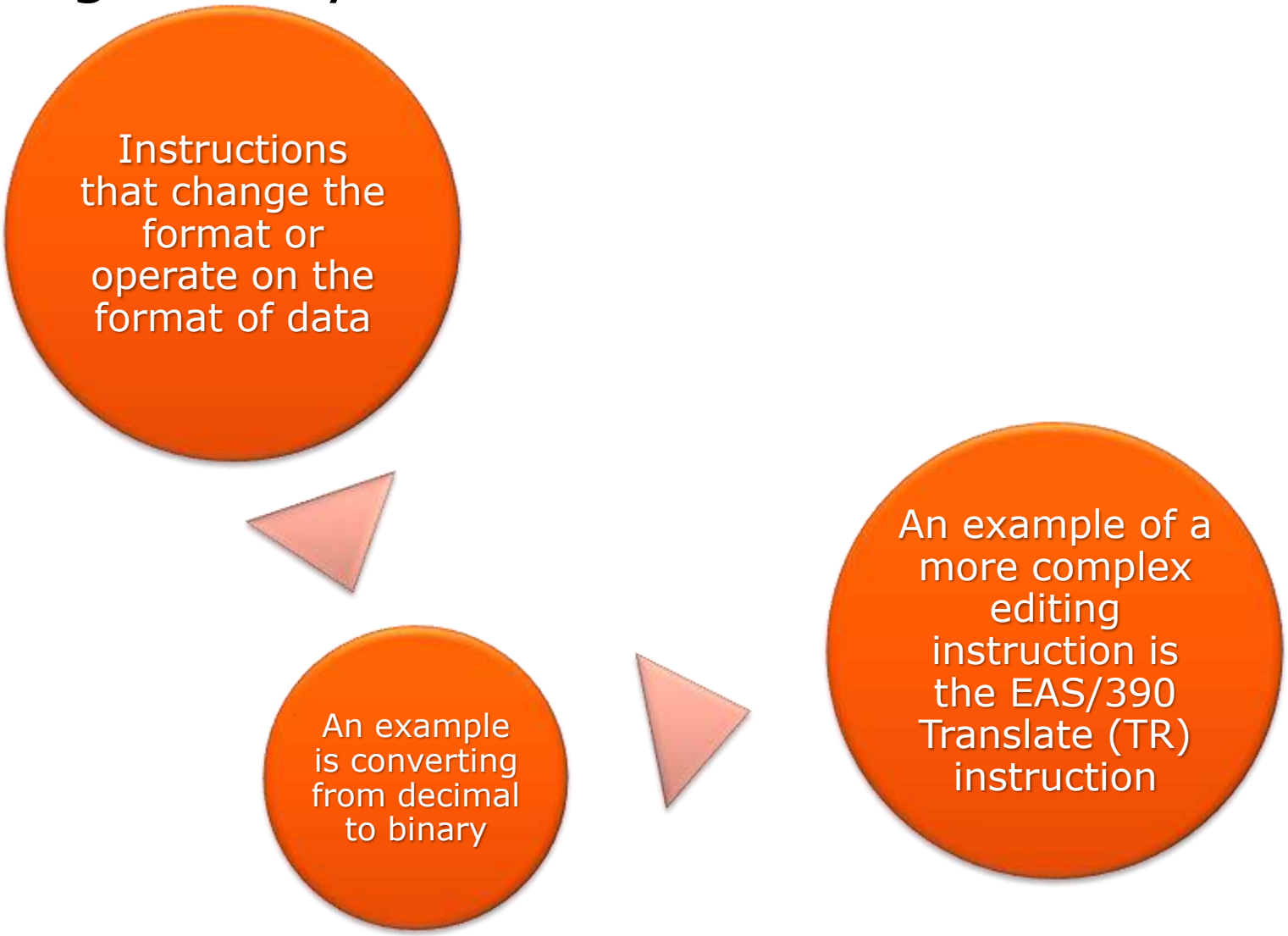
Logical

- Bitwise operations
- AND, OR, NOT

P	Q	NOT P	P AND Q	P OR Q	P XOR Q	P=Q
0	0	1	0	0	0	1
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	1	0	1	1	0	1

Conversion

- E.g. Binary to Decimal

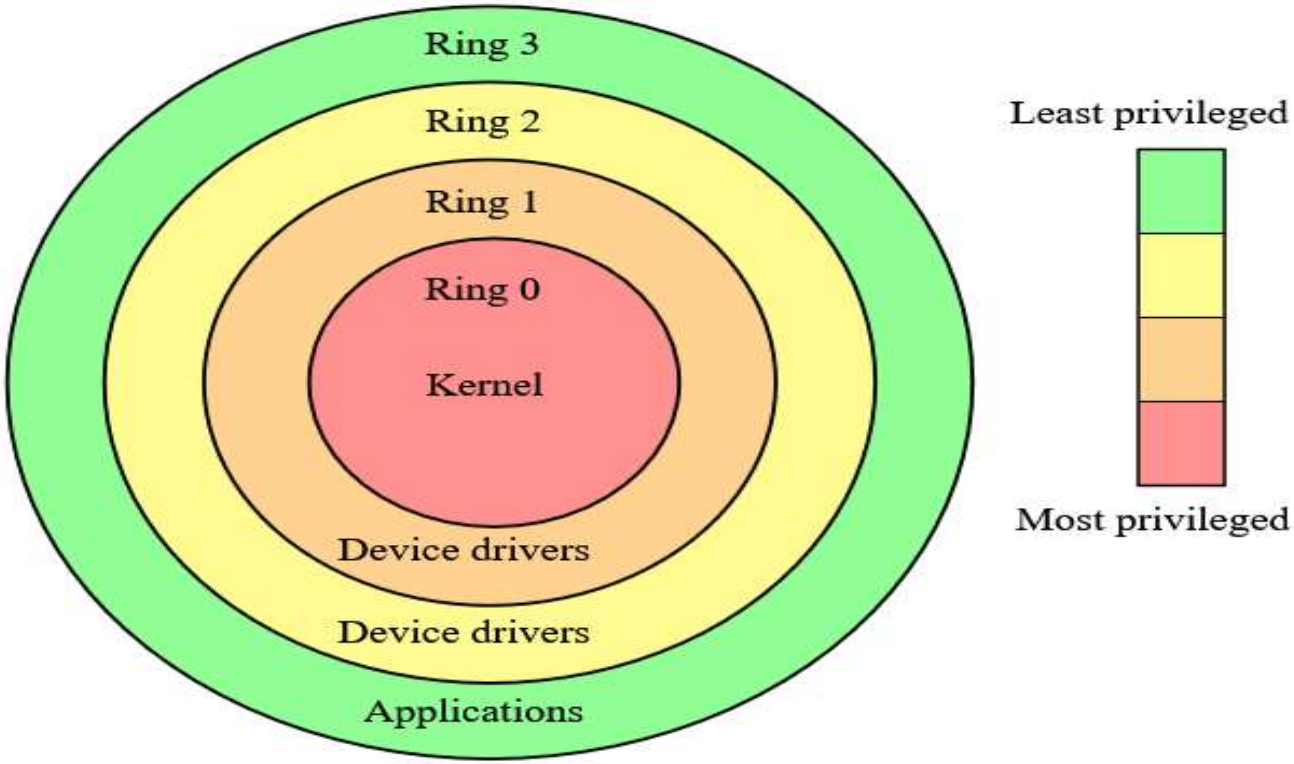


Input/Output

- Isolated programmed I/O: specific instructions
- Memory-mapped programmed I/O: data movement instructions (memory mapped)
- May be done by a separate controller (DMA)
- Use of an I/O processor

Systems Control

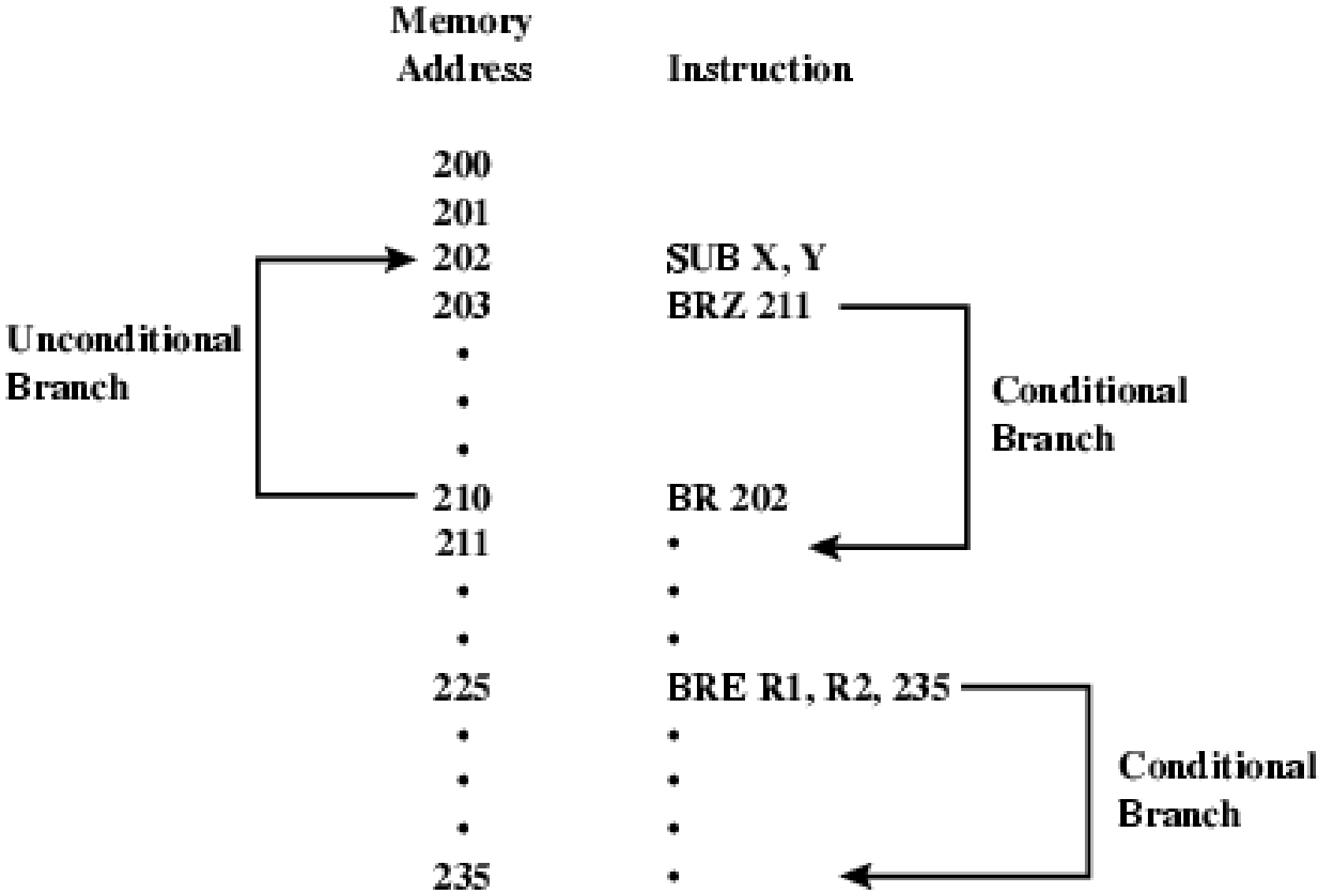
- Privileged instructions
- CPU needs to be in specific state
  - Ring 0 on 80386+
  - Kernel mode
- For operating systems use



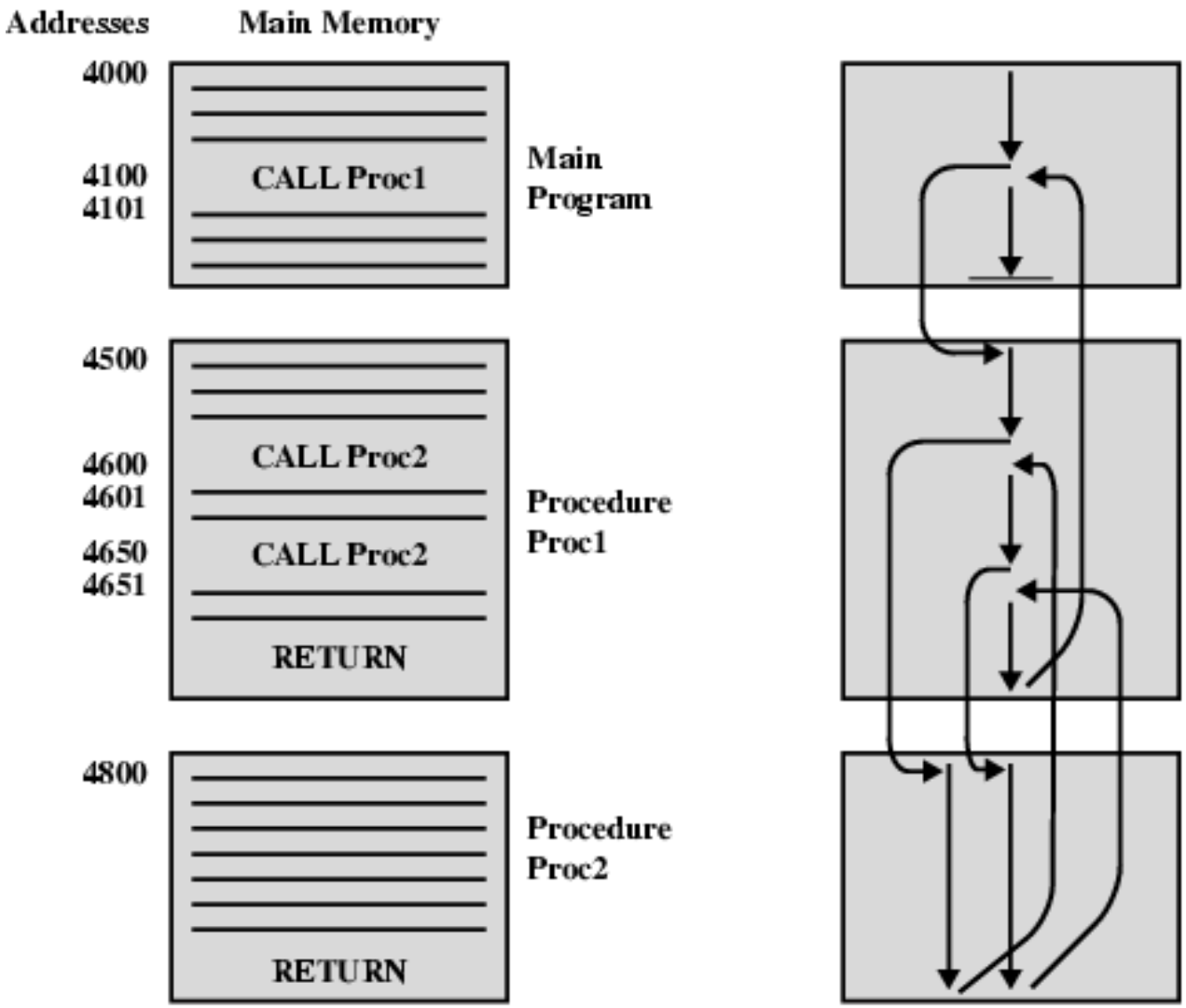
Transfer of Control

- Branch
  - e.g. branch to x if result is zero
- Skip
  - e.g. increment and skip if zero
  - ISZ (Increment and Skip if Zero) Register1
  - Branch xxxx
  - ADD A
- Subroutine call
  - c.f. interrupt call

Branch Instruction



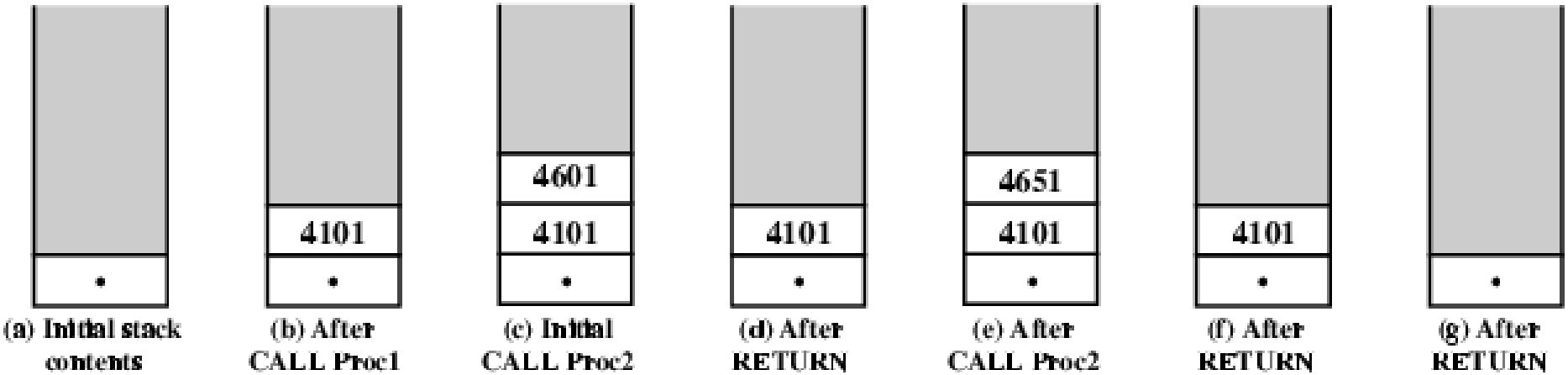
Nested Procedure Calls



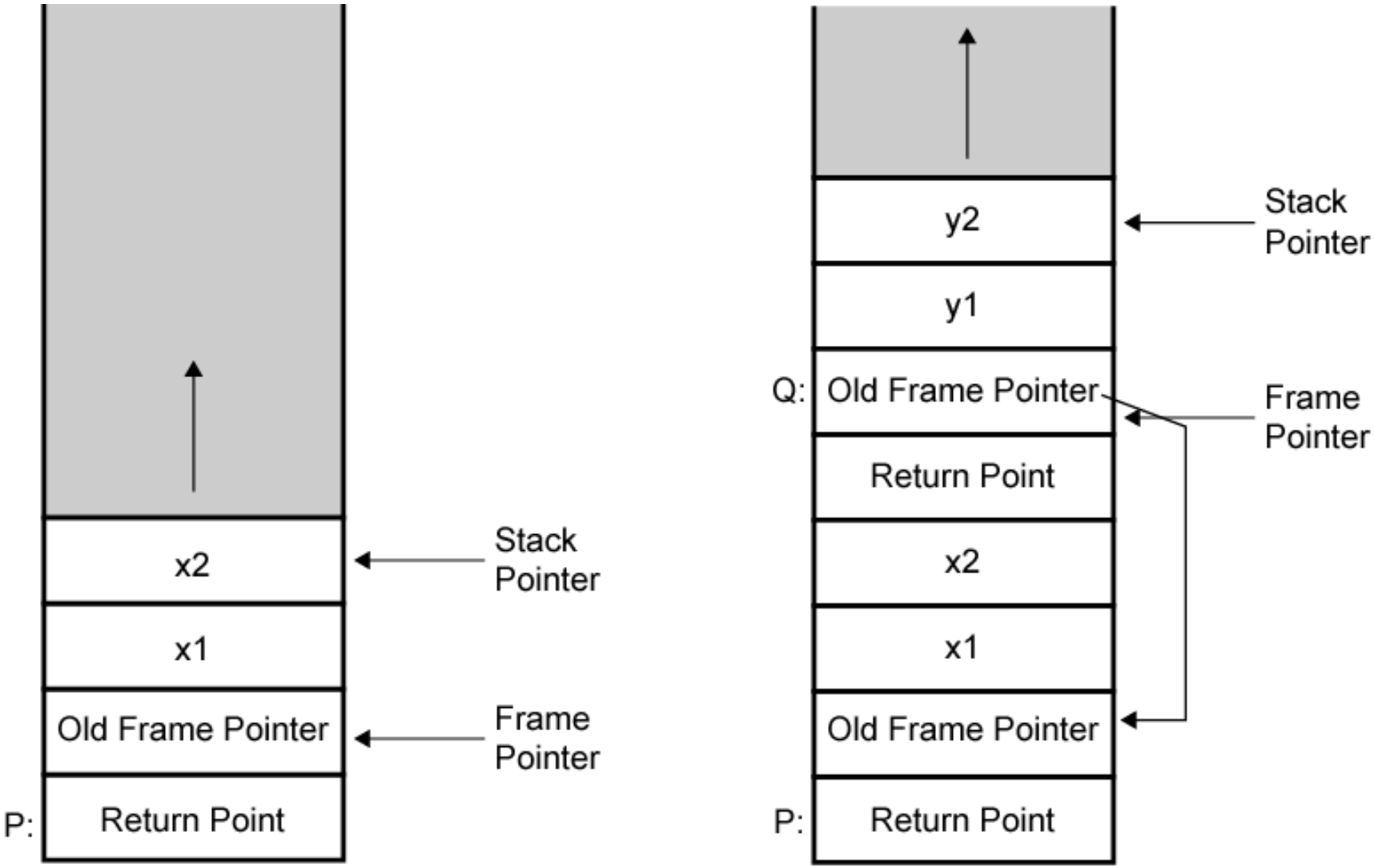
(a) Calls and returns

(b) Execution sequence

Use of Stack


4101
4601
4101
4101
4651
4101
4101
(a) Initial stack contents
(b) After CALL Proc1
(c) Initial CALL Proc2
(d) After RETURN
(e) After CALL Proc2
(f) After RETURN
(g) After RETURN

Stack Frame Growth Using Sample Procedures P and Q



(a) P is active

(b) P has called Q

Byte Order
(A portion of chips?)

- What order do we read numbers that occupy more than one byte
- e.g. (numbers in hex to make it easy to read)
- 12345678 can be stored in 4x8bit locations as follows

Byte Order (example)

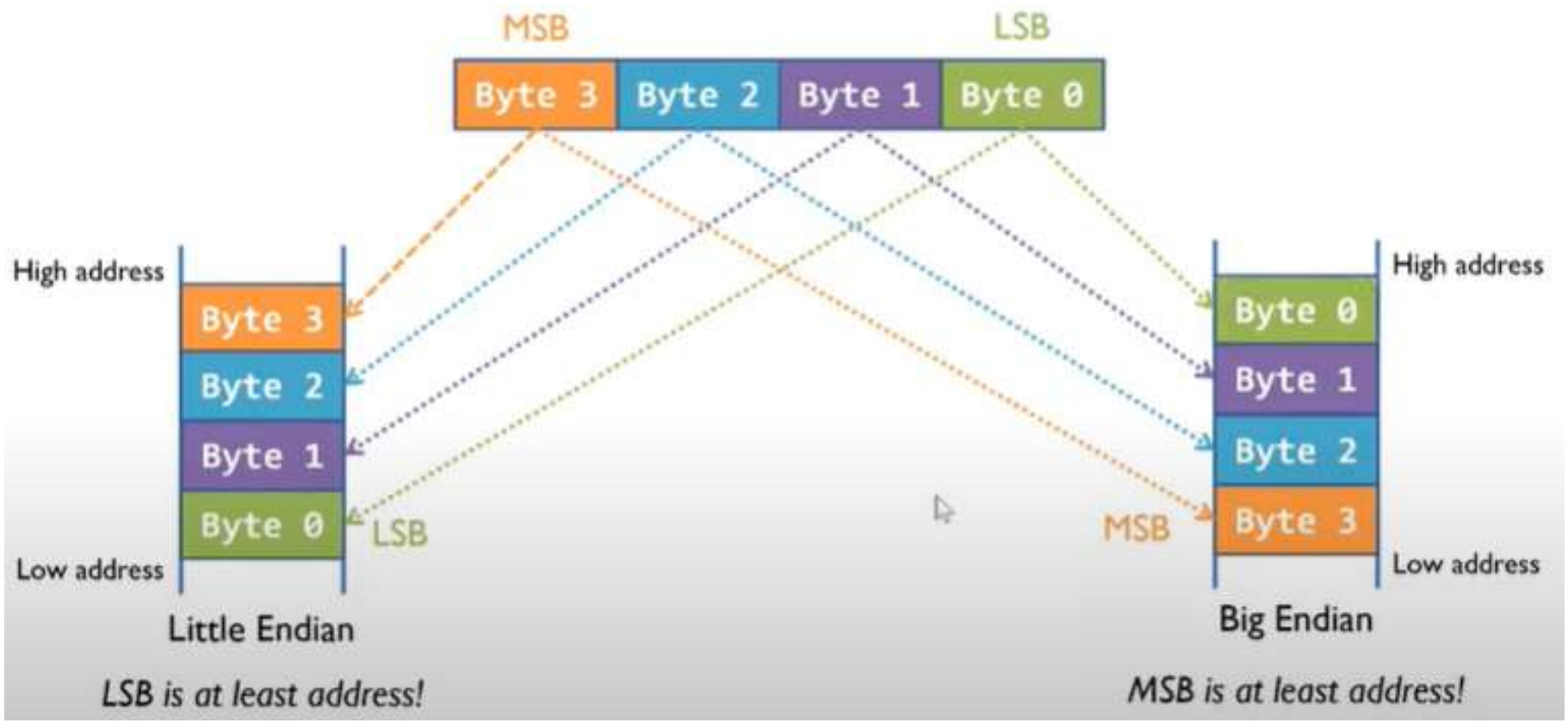
Address	Value (1)	Value(2)
184	12	78
185	34	56
186	56	34
187	78	12

- i.e. read top down or bottom up?

Byte Order Names

- The problem is called Endian
- The system on the right has the Most significant byte in the lowest address
- This is called big-endian
- The system on the left has the least significant byte in the lowest address
- This is called little-endian

Little Endian vs. Big Endian



Standard…What Standard?

- Pentium (80x86), VAX are little-endian
- IBM 370, Moterola 680x0 (Mac), and most RISC are big-endian
- Internet is big-endian
  - Makes writing Internet programs on PC more awkward!
  - WinSock provides htoi and itoh (Host to Internet & Internet to Host) functions to convert